Guarded block transfers for polytope distance: linear convergence, gap-certified screening, and when aggregation pays

Mayank Gupta ORCID[1*]

[1*]Independent Researcher, Seattle, United States.

Corresponding author(s). E-mail(s): iammayankg@gmail.com;

Abstract

Training a hard-margin support vector machine is equivalent to finding the minimum distance between two convex hulls, which the Triangle Algorithm solves with a certified duality gap. We investigate *guarded block transfers*, which aggregate $\boldsymbol{k}$ Mitchell–Demyanov–Malozemov transfers into one exact line search, guarded by a comparison with the best single transfer; if the leading transfer is capacity-clipped, it is replaced by an away step. We establish linear convergence on the Minkowski-difference polytope, with constants depending only on the pyramidal width and diameter, without combinatorial swap-step bounds, and a finite activation bound for gap-safe screening. We then ask when aggregation is beneficial.

The benefit of a block over a single transfer has an exact expression in two quantities the guard computes: the sum of relative pair gains and the loss from combining directions. Under strict complementarity and non-degeneracy, every drop step occurs before an explicitly identified iteration. A fitted work model gives the block size that minimises the per-iteration term, which the solver sets from the measured combination loss. On five LIBSVM problems, aggregation helped only where per-iteration work dominated column computation, and then by at most 1.9 times over an exact-search single transfer. On the separable real dataset the solver tied LIBSVM, but LIBLINEAR's dual solver was up to 4.5 times faster. Our full-cache sequential minimal optimisation code tied or beat the solver on every kernel $\boldsymbol{L_2}$ margin; on the overlapping datasets' $\boldsymbol{L_2}$ margins, the solver was 30 to over 2,000 times slower than LIBLINEAR's Newton solver.

Keywords: polytope distance, Frank–Wolfe methods, Triangle Algorithm, support vector machines, safe screening, linear convergence

1 Introduction

Given two finite point sets $V, W \subset \mathbb{R}^d$, the *polytope distance problem* asks for the closest pair of points of their convex hulls, equivalently for the minimum-norm point of the Minkowski difference $\mathrm{conv}(V) - \mathrm{conv}(W)$. It is one of the oldest problems of computational convexity: Gilbert's procedure (Gilbert, 1966), the Mitchell–Demyanov–Malozemov (MDM) algorithm (Mitchell, Demyanov, & Malozemov, 1974) and Wolfe's minimum-norm-point method (Wolfe, 1976) all solve it by moving weight between vertices, and all are instances of the conditional-gradient scheme of Frank and Wolfe (1956). Kalantari's Triangle Algorithm (TA) began with the convex-hull membership problem and its *distance duality* (Kalantari, 2015), published in this journal; Kalantari (2019) carried it to two hulls, where TA I decides whether the hulls intersect and TA II returns the distance with a certified interval $[\mathrm{LB}, \mathrm{UB}]$ from a pair of parallel supporting hyperplanes, the support-function bound of Gilbert (1966) and Wolfe's stopping test in the two-hull setting. The problem's best-known application is the hard-margin support vector machine (SVM): the optimal separator is the perpendicular bisector of the closest pair and the margin is half the hull distance (Bennett & Bredensteiner, 2000; Keerthi, Shevade, Bhattacharyya, & Murthy, 2000), and the L_2 soft margin, the ν-SVM and the kernel variants are the same problem on augmented, reduced or feature-space hulls (Section 7).

This paper keeps two parts of the Triangle Algorithm and replaces the third. It keeps the distance duality and the certificate: every lower bound below comes from the parallel supporting hyperplanes of Kalantari (2019), and the stopping rule is the certified relative gap. It keeps TA I as the intersection test. It replaces the pivot step, a move toward a single vertex, by the MDM weight transfer from the worst active vertex to the best candidate, which is the pairwise Frank–Wolfe (FW) step of Lacoste-Julien and Jaggi (2015) on the difference polytope, and asks how much of the $O(n)$ scan that selects the transfer can be reused across several transfers without weakening the guarantee. The answer is a *guarded block transfer*: k transfers per scan are aggregated into one exact line search, guarded by an $O(k^2)$ comparison with the single best transfer, and replaced by an away step when the leading transfer is capped. In the terms of the FW literature the method is pairwise FW with the TA certificate, a block step rule and an away fallback; we keep the name enhanced TA (ETA), after Gupta and Kalantari (2026), whose dot-product caching it also retains.

Contributions.

1. **Guarded block transfers** (Section 4): pair the top-k receivers with the worst-k active donors, take one exact line search along the aggregated direction, *guard* it by an $O(k^2)$ comparison with the single MDM step, and fall back to an away step on capped iterations.
2. **Linear convergence with constants explicit in the pyramidal width** (Theorem 6, Corollary 13): on $Z = \mathrm{conv}(V) - \mathrm{conv}(W)$ the guarded block algorithm inherits the pyramidal-width rate of pairwise FW (Lacoste-Julien & Jaggi, 2015) up to a factor four, and the away fallback replaces the combinatorial swap-step bound by drop-step counting with up to k added indices per step (cf. Tsuji, Tanaka, & Pokutta, 2022). The per-step contraction does not depend on k; only

the drop count does, and it costs a factor of at most $2k+1$ in the worst-case exponent, which Theorem 17 confines to an identification phase and which the real runs put at 1.01–1.6.

3. **Screening activation with explicit constants** (Theorem 14): the TA certificate is the FW gap divided by the distance (Lemma 12), so gap-safe screening (Ndiaye, Fercoq, Gramfort, & Salmon, 2017) applies, and we bound the iteration by which it has removed every zero-weight point off the optimal faces.
4. **A regime study on real data** (Section 8): two batteries on five LIBSVM datasets locate the boundary between well-separated hulls, where the certified solver ties LIBSVM and our full-cache sequential minimal optimisation (SMO) code but is beaten by LIBLINEAR's dual solver and, on most kernel L_2 cells, by SMO, and nearly touching hulls or overlapping linear L_2 margins, where it is 30 to over 2,000 times slower.
5. **When aggregation pays** (Sections 6 and 9): (a) the exact gain ratio of a block over the single MDM step at the same iterate, $Q(2-\chi)$ or Q/χ in two observable numbers, Q the summed pair gains relative to the leading pair and χ the ratio of the combined direction's squared norm to its directional derivative (Proposition 15); (b) an identification theorem: under strict complementarity and non-degeneracy all drop steps occur before an explicit iteration and none afterwards (Theorem 17); (c) a conditional iteration bound (Corollary 18) and a timing model with the block size that minimises it; (d) instrumented per-step measurements of every quantity in the model on five real cells; (e) an adaptive block size driven by the measured χ, checked on cells not used to set its constants.

What the method offers that the baselines do not is a certified interval at every iteration and an intersection test. With strengthened baselines (Section 8) it is the fastest solver run on only one real problem, the RBF hard margin of gisette, so this paper's claims are the guarantees and the measured account of when aggregation pays, not speed. Code, proofs, numerical checks and experiment scripts are in the public repository, https://github.com/iammayankg/triangle-algorithm-sv.

2 Related work

Polytope distance in operations research.

Gilbert (1966), Mitchell et al. (1974) and Wolfe (1976) give the classical minimum-norm-point methods; López and Dorronsoro (2015) analyse MDM's convergence, and Kalantari (2015, 2019) develop the Triangle Algorithm and its separating-hyperplane theorem for the convex-hull membership and distance problems. Peña, Rodríguez, and Soheili (2016) relate von Neumann's algorithm and FW with away steps on the same polytope feasibility problem and obtain linear rates through a condition number of the polytope, the closest relative of the constants in Theorem 6.

Frank–Wolfe variants.

The conditional-gradient method of Frank and Wolfe (1956), revisited by Jaggi (2013), converges sublinearly on polytopes; away steps restore a linear rate under strong convexity with an interior-optimum condition (Guélat & Marcotte, 1986) and, for

general polytopes, with the pyramidal width (Lacoste-Julien & Jaggi, 2015); Garber and Hazan (2016) obtain linear rates through a local linear oracle instead, Beck and Shtern (2017) extend the away-step rate to non-strongly convex composite objectives, Peña and Rodríguez (2019) give the facial distance as a computable alternative to the pyramidal width, and Braun et al. (2025) survey the field. Pairwise FW, the MDM transfer, carries a combinatorial swap-step bound that blended pairwise conditional gradients (BPCG) (Tsuji et al., 2022) and the short-step chains of Rinaldi and Zeffiro (2023) remove; boosted FW (Combettes & Pokutta, 2020) aggregates several atoms into one direction by gradient alignment, and block-coordinate FW (Lacoste-Julien, Jaggi, Schmidt, & Pletscher, 2013) blocks over coordinates of a product domain. Our step differs from each of these in one respect, made precise below.

Corrective steps and active-set control.

Fully-corrective FW and Wolfe's method re-optimise over the active set at every iteration (Lacoste-Julien & Jaggi, 2015). Halbey, Rakotomandimby, Besançon, Designolle, and Pokutta (2025) give a corrective-FW framework that covers blended conditional gradients, BPCG and fully-corrective FW, with an optimal-face identification property, and two cheap quadratic corrections, one by linear optimisation and one by a linear system in the spirit of Wolfe's method, for convex quadratic objectives such as this one. The pivoting meta-algorithm of Besançon, Pokutta, and Wirth (2025) keeps the active set within Carathéodory's bound of the dimension plus one, and for away-step FW and BPCG within the dimension of the optimal face plus one. The nearest-extreme-point oracle of Garber and Wolf (2021) yields rates that depend on the dimension of the optimal face rather than the ambient one. These methods act on the correction and the size of the active set; the guarded block acts on how many transfers one scan pays for, and Remark 10 shows that a correction could be added to its guard without changing the rate. Two features of this problem limit what active-set control buys: the side representation already stores $|\operatorname{supp}\alpha| + |\operatorname{supp}\beta|$ weights for the $|\operatorname{supp}\alpha| \cdot |\operatorname{supp}\beta|$ product atoms, and the L_2 augmentation and the kernels lift the problem to dimension $d+n+m$ or to feature space, where Carathéodory's bound does not bind. None of these methods was run here (Section 10).

SVM training as polytope distance.

The nearest-point view of the SVM is due to Keerthi et al. (2000) and Bennett and Bredensteiner (2000), whose reduced convex hulls, the soft convex hulls of Crisp and Burges (2000) and the geometric formulation of Mavroforakis and Theodoridis (2006) give the ν-SVM, with the iterative maximal-margin algorithm of Franc and Hlaváč (2003); core-set and FW solvers for the same problem are Clarkson (2010); Gärtner and Jaggi (2009); Ñanculef, Frandi, Sartori, and Allende (2014); Tsang, Kwok, and Cheung (2005). The production solvers we compare against are SMO (Platt, 1999) as implemented in LIBSVM (Chang & Lin, 2011) and LIBLINEAR's primal trust-region Newton and dual coordinate-descent solvers (Fan, Chang, Hsieh, Wang, & Lin, 2008; Hsieh, Chang, Lin, Keerthi, & Sundararajan, 2008), all through scikit-learn (Pedregosa et al., 2011), together with our own implementations of SMO, BPCG, away-step FW and single-transfer MDM.

Screening and identification.

Gap-safe rules (Fercoq, Gramfort, & Salmon, 2015; Ndiaye et al., 2017; Shibagaki, Karasuyama, Hatano, & Takeuchi, 2016) screen with a radius from the duality gap and identify the equicorrelation set, a superset of the support, in finite time; the active-set complexity of away-step FW (Bomze, Rinaldi, & Zeffiro, 2020) and the strict-complementarity bounds of Garber (2020) give identification from the algorithm's side. Ogawa, Suzuki, and Takeuchi (2013) adapted the feature-elimination rules of El Ghaoui, Viallon, and Rabbani (2012) to the screening of non-support vectors, and Wang, Wonka, and Ye (2014) reduce the data exactly. The rule of Section 4 is the gap-safe sphere of Ndiaye et al. (2017) with the TA certificate as its dual pair; what is new in Theorem 14 is the activation bound, and Theorem 17 is the corresponding identification statement.

What is different here.

Boosted FW builds its direction by greedily aligning a combination of atoms with the negative gradient, at several oracle calls per iteration, and inherits the FW rate through an alignment condition; our block fixes the pair coefficients γ_j from one scan and relies on the guard, not on alignment, for its rate. BPCG removes pairwise FW's swap-step bound with a guard between the pairwise step and a FW step; our guard compares an aggregated block of k transfers with the single MDM step and falls back to an away step, so the drop-step count of away-step FW carries over with up to k (two-sided: $2k$) indices added per non-drop step, and each non-drop case-(b) step is at least as good as the best of the block, MDM, away and toward candidates. BPCG's linear rate has the same pyramidal-width form as Theorem 6; it is compared on the same real cells in Table 13. Block-coordinate FW and the short-step chains of Rinaldi and Zeffiro (2023) act on the domain decomposition and on the step length, not on the direction. Wolfe's method minimises over the affine hull of a corral at every iteration, which needs a different oracle and implementation, and was not run. A bound on the pyramidal width of the difference atom set $A = \{v_i - w_j\}$ in terms of the two class hulls is open, and none can depend on the class hulls' conditioning alone: $V = \{0, e_1\}$ and $W = \{0, -(e_1 + \epsilon e_2)\}$, translated apart, have well-conditioned class hulls while the four difference atoms span a parallelogram of height ϵ, so that $\mathrm{PW}(A) \le \epsilon$.

3 Setting

Let $V = \{v_1, \dots, v_n\}$, $W = \{w_1, \dots, w_m\} \subset \mathbb{R}^d$. The polytope distance problem is

$$\min_{p \in \mathrm{conv}(V),\ q \in \mathrm{conv}(W)} \ \tfrac{1}{2}\|p - q\|^2. \tag{1}$$

Writing $z = p - q$ and $Z := \mathrm{conv}(V) - \mathrm{conv}(W) = \mathrm{conv}\{v_i - w_j\}$, (1) is the minimum-norm-point problem $\min_{z \in Z} F(z)$, $F(z) = \frac{1}{2}\|z\|^2$, whose objective is 1-strongly convex and 1-smooth. Let z^* be the minimiser, $\delta^* = \|z^*\|$, $h(z) = F(z) - F(z^*)$, $M = \mathrm{diam}(Z)$ and $\delta = \mathrm{PW}(A) > 0$ the pyramidal width (Lacoste-Julien & Jaggi, 2015) of the atom set $A = \{v_i - w_j\}$ (a property of the atom set, not of Z alone).

The algorithm keeps simplex weights α on V, β on W (iterate z_t after t iterations), with $p = \sum \alpha_i v_i$, $q = \sum \beta_j w_j$; the induced product representation $\lambda_{ij} = \alpha_i \beta_j$ of z over the atoms $v_i - w_j$ has support $\operatorname{supp}\alpha \times \operatorname{supp}\beta$. With $g = -\nabla F(z) = q - p$, the *side scores* are $s_i^V = \langle g, v_i \rangle$ and $s_j^W = \langle p - q, w_j \rangle$, maintained incrementally from cached arrays.

Certificate.

For any iterate, $\mathrm{UB} = \|z\|$ and $\mathrm{LB} = \min_{u \in Z} \langle z, u \rangle / \|z\| = \big(\min_i \langle z, v_i \rangle - \max_j \langle z, w_j \rangle\big)/\mathrm{UB}$ satisfy $\mathrm{LB} \le \delta^* \le \mathrm{UB}$; the TA stops when the certified relative gap $(\mathrm{UB} - \mathrm{LB})/\mathrm{UB}$ is at most ε. The L_2 soft margin with regularisation parameter C, the ν-SVM and the kernel variants reduce to (1) exactly (augmented coordinates that turn the Gram matrix K into $K + I/C$; reduced convex hulls; feature-space images), see Section 7.

Gaps.

$g^{\mathrm{FW}} = \max_{u \in Z} \langle g, u \rangle - \langle g, z \rangle$, $g^{\mathrm{A}} = \langle g, z \rangle - \min_{u \in \operatorname{supp}\lambda} \langle g, u \rangle$, $g^{\mathrm{PW}} = g^{\mathrm{FW}} + g^{\mathrm{A}}$. Because $\langle g, v_i - w_j \rangle = s_i^V + s_j^W$ is separable and the support is a product set, each gap splits into a V-side and a W-side term computable from the score arrays in $O(n+m)$ (Lemma 1), written $g_V^{\mathrm{PW}}, g_W^{\mathrm{PW}}$ (likewise for FW and A), with $g_{\mathrm{side}}^{\mathrm{PW}}$ the term of the side being stepped; a transfer $v_r - v_u$ on the V side is a pairwise direction of Z (Lemma 2).

Lemma 1 (Separability) *With the product representation, $g^{\mathrm{PW}} = g_V^{\mathrm{PW}} + g_W^{\mathrm{PW}}$ where $g_V^{\mathrm{PW}} = \max_i s_i^V - \min_{i \in \operatorname{supp}\alpha} s_i^V \ge 0$ and g_W^{PW} likewise; the same holds for g^{FW} and g^{A}.*

Proof $\langle g, v_i - w_j \rangle = s_i^V + s_j^W$ is separable and the support is a product set, so the max over atoms and the min over the support separate. Each side term is nonnegative: the FW term is the maximum score minus the weighted mean of the support scores, the away term is that mean minus the support minimum, and a weighted mean lies between the minimum and the maximum. □

Lemma 2 (Side transfers are Z-pairwise directions) *For $\alpha_u > 0$ and any $b \in \operatorname{supp}\beta$, $v_r - v_u = (v_r - w_b) - (v_u - w_b)$ is a difference of an atom of Z and an atom in the support, and the transfer with $0 \le \gamma \le \alpha_u$ realises $z \leftarrow z + \gamma(v_r - v_u)$ inside Z. The side capacity $\alpha_u = \sum_b \alpha_u \beta_b$ is the total weight of the product atoms $v_u - w_b$, $b \in \operatorname{supp}\beta$: the side transfer is the sum of the atom-pair transfers of masses $\gamma\beta_b$ from $v_u - w_b$ to $v_r - w_b$, and for $r \ne u$ this weight update is feasible exactly when $0 \le \gamma \le \alpha_u$; hence every such transfer remains in Z.*

Table 1 Notation. Symbols with a second, conventional meaning in one place only are listed with both.

V, W; n, m; d	point sets, their sizes, dimension	Z, A	$\operatorname{conv}(V) - \operatorname{conv}(W)$; its atoms $v_i - w_j$
α, β	simplex weights on V, W	$z = p - q$, z^*, δ^*	iterate, optimum, $\|z^*\|$
h, h_t	$F(z) - F(z^*)$, at iteration t	M, R, L	$\operatorname{diam} Z$, $\max_Z \|u\|$, larger class diameter
δ, ρ	pyramidal width of A; rate $(\delta/M)^2/16$	$g^{\mathrm{FW}}, g^{\mathrm{A}}, g^{\mathrm{PW}}$	Frank–Wolfe, away, pairwise gaps
s_i^V, s_j^W	side scores	UB, LB, ε	certificate bounds, relative tolerance
k, k'	requested, retained block size	(r_j, u_j), d_j	receiver–donor pair j, $v_{r_j} - v_{u_j}$
gap_j, γ_j	pair gap, transfer length	S, D, θ	$\langle g, d_B \rangle$, $\|d_B\|^2$, line-search step
$\Delta_B, \Delta_1, \Delta_j$	block, MDM, pair-j gains	Q, χ	$\sum_j \Delta_j/\Delta_1$, D/S
η, $\hat{\eta}$	max pair cosine; $(\chi - 1)/(k' - 1)$	β_{flat}	flatness $\min_j \Delta_j/\Delta_1$
ℓ_t, ℓ_0	total support size, initial (2)	ζ_k	$k+1$ (one-sided), $2k+1$ (two-sided)
ϱ, ϱ_t	screening radius $\sqrt{\mathrm{UB}^2 - \max(\mathrm{LB}, 0)^2}$	τ_i, τ	score margin of v_i; smallest nonzero margin
ϖ_t	$\|z_t - z^*\|$ $(\leq \sqrt{2h_t})$	ω_t	weight on non-face points
d_B, Γ	block direction $\sum \gamma_j d_j$; pair Gram matrix	g, p, q	$-\nabla F(z) = q - p$; class iterates
ρ_t^{act}, $\underline{\rho}_j$	actual contraction; its bound for j pairs	k_t^{ev}, $t_1(k)$, h_3	pairs evaluated; end of identification; threshold term
F_V, F_W; $\mathcal{F}_V, \mathcal{F}_W$	optimal face index sets; faces	α^*, β^*, $\alpha_{\min}$	optimal weights; smallest positive one
σ, $d_{\min}$	weight-recovery constant, vertex-to-hull distance	ϑ_V, ϑ_W	$\min_i \langle z^*, v_i \rangle$, $\max_j \langle z^*, w_j \rangle$
H, H_{id}, H_{w}	activation, identification thresholds	$t_{\mathrm{id}}, t_{\mathrm{w}}$, N	their iterations; non-face support count
a, b, c_{col}	fitted timing coefficients	$\mathcal{C}_B, \mathcal{C}_1$; n^{new}	step costs; column misses
k^*	$\sqrt{(a/b)(1-\eta)/\eta}$ (reported with $\hat{\eta}$)	N_{cols}, $W(k)$	columns computed; modelled work
C	L_2 regularisation parameter	$\kappa(\cdot, \cdot)$, K	kernel, Gram matrix
$\mathcal{R}(V, \mu)$, ν	reduced hull, ν-SVM parameter	w, b	separator normal and bias (Sections 7 and 8)
λ_{ij}, μ	product weights $\alpha_i \beta_j$; reduced-hull cap (Section 7)	α (LIBSVM)	dual coefficients, Section 7 only

4 Guarded block transfers

One iteration on side V (side W is symmetric) starts from the score scan of Section 3. It pairs the k receivers of highest score with the k donors of lowest score in $\operatorname{supp}\alpha$, best with worst, and keeps the $k' \le k$ pairs (r_j, u_j) with positive score gap; pair 1 is the MDM pair (r^*, u^*). Each pair defines a transfer direction $d_j = v_{r_j} - v_{u_j}$ with gap $\mathrm{gap}_j = s^V_{r_j} - s^V_{u_j}$ and a transfer amount γ_j, the unconstrained one-dimensional optimum $\mathrm{gap}_j/\|d_j\|^2$ clipped to the donor's weight α_{u_j}; a pair whose amount is clipped is *capped*. The block direction is $d_B = \sum_j \gamma_j d_j$ with $\gamma = (\gamma_1, \dots, \gamma_{k'})^\top$; receivers that are themselves donors are discarded. Algorithm 1 states the step, its guard and its fallback.

Algorithm 1 amortises one $O(n)$ scan over up to k transfers. Only the scan is amortised: a new receiver still costs one Gram or kernel column, $O(nd)$. Every candidate gain is available from cached quantities at a cost independent of n: with $S = \langle g, d_B\rangle = \sum_j \gamma_j \mathrm{gap}_j$ and $D = \|d_B\|^2 = \gamma^\top \Gamma \gamma$, $\gamma = (\gamma_1, \dots, \gamma_{k'})^\top$ and Γ the $k' \times k'$ Gram matrix of the pair directions, assembled from k'^2 cached entries, the block gain is $\theta S - \theta^2 D/2$ at $\theta = \min(S/D, 1)$; the MDM, away and toward gains are the usual one-dimensional expressions. The *gain* of a step is the decrease $h(z_t) - h(z_{t+1})$. The cache update for a block is one matrix–vector product with the net per-index coefficients; columns are computed on first use and cached, so support columns cost nothing after their first appearance. Feasibility is immediate, since donors lose at most $\theta\gamma_j \le \alpha_{u_j}$. We call an iteration *case (a)* if pair 1 is uncapped and *case (b)* otherwise; the boundary-clipped away step of case (b), which drives α_{u^*} to zero, is a *drop* step (a capped block pair may also zero a donor, which only helps the counts below).

Safe screening.

With $\varrho = \sqrt{\mathrm{UB}^2 - \max(\mathrm{LB}, 0)^2}$ and $i_{\min} \in \arg\min_i \langle z, v_i\rangle$, $v_{\min} = v_{i_{\min}}$, a zero-weight point with $\langle z, v_i\rangle - \langle z, v_{\min}\rangle > \varrho\|v_i - v_{\min}\|$ is outside the optimal support and is removed (symmetrically on W with the max). Because the optimum's support survives, the reduced problem has the same optimum and all subsequent lower bounds remain valid.

Algorithm 1: One iteration of the guarded block-transfer step (side V; W symmetric)

top-k receivers by s^V; worst-k donors in $\operatorname{supp}\alpha$; pair best-with-worst: (r_j, u_j), $j \le k'$ ($k' \le k$ pairs, limited by the support size); pair 1 is the MDM pair (r^*, u^*)
$d_j \leftarrow v_{r_j} - v_{u_j}$; $\mathrm{gap}_j \leftarrow s^V_{r_j} - s^V_{u_j}$; discard pairs with $\mathrm{gap}_j \le 0$ and receivers that are donors (if r^* is a donor it leaves the donor list, so pair 1 survives); $\gamma_j \leftarrow \min(\mathrm{gap}_j/\|d_j\|^2,\ \alpha_{u_j})$ `// $\alpha_{u_j}$: donor capacity`
if *pair 1 uncapped ($\gamma_1 = \mathrm{gap}_1/\|d_1\|^2$)* [case (a)] **then**
  $\Delta_B \leftarrow$ gain of exact line search $\theta \in [0,1]$ along $d_B = \sum_j \gamma_j d_j$; $\Delta_1 \leftarrow$ gain of the MDM step
  take the step with the larger gain `// guard, $O(k'^2)$`
else
  `// case (b): pair 1 capped`
  if *the away step on u^* is boundary-clipped* **then**
    take it `// drop step: $\alpha_{u^*} \to 0$`
  else
    take the best of {block, MDM, away(u^*), toward(r^*)}
  end
end

5 Analysis

Conventions.

$M > 0$ (a singleton Z is solved at initialisation), and TA II stops with "intersect" when $\mathrm{UB} = \|z\|$ falls below the intersection tolerance (10^{-6} in the implementation), before any division by UB; TA I's verdict is likewise an ε-witness (Section 8). Lemma 3 and Algorithm 1 presuppose $k' \ge 1$: a side with no positive-gap pair is skipped, and on a non-drop iteration the side with the larger pairwise gap has one. An away direction whose donor has weight one is unavailable (the iterate is that vertex and the away gap of that side is zero).

Lemma 3 (Block gain) *Let $\varphi_j = \gamma_j\|d_j\|$ over the k' retained pairs, all with $\mathrm{gap}_j > 0$. Then $S \ge \sum_j \varphi_j^2$, $D \le k'S$, and the realised block gain satisfies $\Delta_B \ge S/(2k') \ge \Delta_1/(2k')$, Δ_1 the MDM gain.*

Proof of Lemma 3 Since $\gamma_j \le \mathrm{gap}_j/\|d_j\|^2$, we have $\gamma_j\mathrm{gap}_j \ge \gamma_j^2\|d_j\|^2 = \varphi_j^2$. Consequently, $S = \sum_j \gamma_j\mathrm{gap}_j \ge \sum_j \varphi_j^2$. By the triangle inequality, $\|d_B\| \le \sum_j \gamma_j\|d_j\| = \sum_j \varphi_j$, and by Cauchy–Schwarz, $(\sum_j \varphi_j)^2 \le k' \sum_j \varphi_j^2 \le k'S$; hence $D \le k'S$. Exact line search on the 1-smooth F gives $\Delta_B = S^2/(2D) \ge S/(2k')$ in the interior case and $\Delta_B = S - D/2 \ge S/2$ at the boundary $\theta = 1$, where $D \le S$. Finally, $S \ge \gamma_1\mathrm{gap}_1 \ge \Delta_1$, since all terms are nonnegative and $\Delta_1 = \gamma_1\mathrm{gap}_1 - \gamma_1^2\|d_1\|^2/2$. □

With the guard, every case-(a) iteration gains $\max(\Delta_B, \Delta_1) \geq \Delta_1$, at least the MDM gain; a non-drop case-(b) iteration gains at least the best single candidate, and drop iterations, which are taken without comparison, are counted separately in Theorem 6.

Lemma 4 (Good-step progress) *If the iteration works on the side with the larger pairwise gap, then on any non-drop iteration* gain $\geq \min\big((g^{\mathrm{PW}})^2/(32M^2),\ g^{\mathrm{PW}}/8\big)$.

Proof of Lemma 4 Let $g^{\mathrm{PW}}_{\mathrm{side}} = \max(g^{\mathrm{PW}}_V, g^{\mathrm{PW}}_W) \geq g^{\mathrm{PW}}/2$. Every direction below is a difference of two points of one class hull, so its norm is at most $\mathrm{diam}(\mathrm{conv}\, V) \leq M$. For a generic feasible direction with directional derivative $\varsigma > 0$, squared norm D' and maximal step $\bar{\theta}$, exact line search on a 1-smooth function gives gain $\geq \min(\varsigma^2/(2D'), \bar{\theta}\varsigma/2)$ (interior optimum, or the clipped step evaluated with $\bar{\theta}D' \leq \varsigma$). In case (a), the guard's gain is at least the MDM gain, whose derivative is $g^{\mathrm{PW}}_{\mathrm{side}}$ and whose interior optimum is attained, yielding gain $\geq (g^{\mathrm{PW}}_{\mathrm{side}})^2/(2M^2) \geq (g^{\mathrm{PW}})^2/(8M^2)$. In case (b), non-drop, $g^{\mathrm{PW}}_{\mathrm{side}} = g^{\mathrm{FW}}_{\mathrm{side}} + g^{\mathrm{A}}_{\mathrm{side}}$, so one term is at least $g^{\mathrm{PW}}_{\mathrm{side}}/2 \geq g^{\mathrm{PW}}/4$. If it is the FW term, the toward step (maximal step 1) gives gain $\geq \min((g^{\mathrm{FW}}_{\mathrm{side}})^2/(2M^2), g^{\mathrm{FW}}_{\mathrm{side}}/2) \geq \min((g^{\mathrm{PW}})^2/(32M^2), g^{\mathrm{PW}}/8)$; if it is the away term, the away line search is interior (non-drop) and gain $\geq (g^{\mathrm{A}}_{\mathrm{side}})^2/(2M^2) \geq (g^{\mathrm{PW}})^2/(32M^2)$. The algorithm takes the best available candidate. □

Lemma 5 (Geometric strong convexity; Lacoste-Julien & Jaggi, 2015, Thm. 3 with $\mu = 1$) *For every $z \in Z$ written as a proper convex combination of an active set of atoms, with g^{A} and g^{PW} evaluated over that active set (the product representation of Section 3 is one), $g^{\mathrm{PW}}(z) \geq \delta\sqrt{2h(z)}$ and $g^{\mathrm{PW}}(z) \geq g^{\mathrm{FW}}(z) \geq h(z)$.*

Proof This is the pyramidal-width argument of Lacoste-Julien and Jaggi (2015, Thm. 3) for a 1-strongly convex objective; we include the short version. If $z = z^*$ both inequalities are immediate ($h = 0$ and all gaps are nonnegative). Otherwise let $e = (z^* - z)/\|z^* - z\|$; the pyramidal-width inequality, eq. (10) of Lacoste-Julien and Jaggi (2015), gives, for the FW atom u^{FW} and the away atom u^{A}, $\langle g, u^{\mathrm{FW}} - u^{\mathrm{A}}\rangle \geq \delta\langle g, e\rangle$. By 1-strong convexity, $\langle g, z^* - z\rangle \geq h + \frac{1}{2}\|z^* - z\|^2$, so with $\upsilon = \|z^* - z\|$, $g^{\mathrm{PW}} \geq \delta(h/\upsilon + \upsilon/2) \geq \delta\sqrt{2h}$ by AM–GM. Finally $g^{\mathrm{PW}} = g^{\mathrm{FW}} + g^{\mathrm{A}} \geq g^{\mathrm{FW}} = \max_u \langle g, u - z\rangle \geq \langle g, z^* - z\rangle \geq h$ by convexity. □

Theorem 6 (Global linear convergence) *Suppose the guarded block-transfer algorithm with away fallback is run for one side per iteration (the side with the larger pairwise gap), using any $k \geq 1$, from any vertex pair (initial support $\ell_0 = 2$). Let $h_t = h(z_t)$. Then every non-drop iteration satisfies $h_{t+1} \leq (1 - \rho)h_t$ with*

$$\rho = \tfrac{1}{16}\,(\delta/M)^2.$$

For any horizon T, among the first T iterations the number of drop steps is at most $k\,T_{\mathrm{good}} + \ell_0$, where T_{good} is the number of non-drop iterations; consequently, $h_T \leq h_0\,(1 - \rho)^{(T-\ell_0)/(k+1)}$. Under the two-sided schedule of the implementation (larger-gap side first, second side skipped after a drop) every non-drop iteration has the same contraction, the drop count is at most $2kT_{\mathrm{good}} + \ell_0$, and $h_T \leq h_0\,(1 - \rho)^{(T-\ell_0)/(2k+1)}$ (Lemma 8).

The rate combines Lemma 4 with $g^{\mathrm{PW}} \geq \delta\sqrt{2h}$ and $g^{\mathrm{PW}} \geq g^{\mathrm{FW}} \geq h$ (Lemma 5). Drop counting is the away-step argument: each drop removes one support index and non-drop steps add at most k. No swap-step bound is needed: a case-(b) iteration either drops (counted above) or takes the best of four candidates, of which the toward or away step alone gains Lemma 4's amount, whichever wins. The constant is a quarter of the case-(a) bound: there the guard's gain is at least $(g^{\mathrm{PW}})^2/(8M^2) \geq 4\rho h$, the $\frac{1}{4}(\delta/M)^2$ rate of away-step FW in Lacoste-Julien and Jaggi (2015) (their μ/L is 1 here), and the factor is lost in case (b), where the toward and away candidates are one-sided. The theorem does not need $\delta^* > 0$: nothing in Lemmas 4 and 5 uses it, and when the hulls intersect $h_t = \frac{1}{2}\|z_t\|^2$ contracts at the same rate, which is what drives TA II to its intersection tolerance; only the certificate conversion of Corollary 13 and Theorems 14 and 17 need $\delta^* > 0$.

Proof of Theorem 6 By Lemmas 4 and 5 a non-drop iteration gains at least

$$\min\left(2\delta^2 h/(32M^2),\, h/8\right) = (\delta/M)^2 h/16$$

using $\delta \leq M$; hence $h_{t+1} \leq (1-\rho)h_t$. Drop counting: non-drop iterations add at most k support indices, drop iterations add none and remove at least one, and the support size stays ≥ 2 from ℓ_0; so the number of removals, hence of drops, is at most $kT_{\mathrm{good}} + \ell_0$, so that

$$T_{\mathrm{good}} \geq (T - \ell_0)/(k+1).$$

All steps, drops included, are descent steps, so h is monotone. □

Remark 7 (What the factor ζ_k costs) The factor $k+1$ (two-sided: $2k+1$) in the exponent is a worst case for the drop count, not a slower contraction: every non-drop iteration contracts by the same k-independent ρ, and the bound charges each of them for k (or $2k$) later drops. Whether that charge is paid is measurable, since $T/T_{\mathrm{good}} \leq T/(T - T_{\mathrm{drop}})$ is the factor the drops actually cost. In the isolated runs at k=16 (Table 12), drops are 36% of the iterations on gisette and 0.8–8.3% on the four kernel cells, so the realised factor is at most 1.6 and 1.01–1.09 against the worst case of 33. Theorem 17 (Section 6, stated and proved in full) explains why: under strict complementarity and non-degeneracy every drop occurs before an explicit iteration, after which the iteration count no longer depends on drops (Corollary 18).

Lemma 8 (Two-sided schedule) *Let every iteration compute both side gaps, take the guarded step of Algorithm 1 on the side with the larger pairwise gap first, and take the other side's guarded step only if the first step was not a drop; an iteration counts as a drop iff its first step is a drop. Then every non-drop iteration gains at least* $\min((g^{\mathrm{PW}})^2/(32M^2),\, g^{\mathrm{PW}}/8)$ *and the number of drops is at most* $2kT_{\mathrm{good}} + \ell_0$*; hence Theorem 6 holds for this schedule with the same* ρ *and exponent* $(T - \ell_0)/(2k+1)$.

Proof On a non-drop iteration the first step is exactly the step analysed in Lemma 4, so it gains at least the stated amount; the second step is an exact line search along a feasible descent direction (or is skipped), so it cannot increase h. Drop counting: a drop iteration takes a single away step that removes one support index and adds none, because the second side is skipped; a non-drop iteration takes two guarded steps and adds at most k indices each. Removals, hence drops, therefore number at most $2kT_{\mathrm{good}} + \ell_0$. □

Remark 9 The skip is what makes the count work. If the second side always steps, an iteration whose analysed side drops can add up to k indices on the other side, and the count becomes $T_{\text{drop}} \le 2kT_{\text{good}} + kT_{\text{drop}} + \ell_0$, which is vacuous for $k \ge 1$. Table 8 compares the two schedules on synthetic instances: the unconditional order takes between 3% fewer and 15% more iterations and is within 8% in time on the two L_2 instances, so the skip is needed for the proof, not for speed. The one-sided schedule of Theorem 6, which has the smaller factor $k+1$, was not benchmarked.

The schedule as implemented.

Every iteration computes both side gaps from the score arrays; a side whose best score is within 10^{-11} of its worst active score is skipped; the side with the larger pairwise gap takes the step of Algorithm 1; the other side takes its guarded step unless that first step was a drop. Screening (Section 4) runs between iterations and removes only zero-weight points, so it changes neither the supports nor the drop count, and compaction keeps the cached columns of the surviving points. Nothing is warm-started across runs.

Remark 10 (More candidates in the guard) Theorem 6 survives any enlargement of the candidate set by steps that keep the iterate in Z, do not increase h and add no support index, for instance a quadratic correction that re-optimises the weights over the current supports, as in Wolfe's method or the corrections of Halbey et al. (2025). In case (a) the guard's gain is still at least Δ_1, in case (b) the best candidate still gains at least the amount of Lemma 4, and the drop count is unchanged, since such a step adds no index and any index it removes only lowers the support. Theorem 17 uses the specific candidates in its proof and would have to be rechecked. We have not implemented this variant.

Proposition 11 (Near-orthogonal speed-up) *If all k' pairs are uncapped and, for some $\eta \ge 0$, $|\langle d_i, d_j\rangle| \le \eta\|d_i\|\|d_j\|$ for $i \ne j$, then the block gain is at least $(\sum_j \Delta_j)/(1+\eta(k'-1))$, where Δ_j is pair j's individual gain.*

Proof of Proposition 11 $D = \sum_{i,j}\gamma_i\gamma_j\langle d_i, d_j\rangle \le \sum_j \varphi_j^2 + \eta\sum_{i\ne j}\varphi_i\varphi_j \le (1+\eta(k'-1))\sum_j \varphi_j^2 \le (1+\eta(k'-1))S$ by Cauchy–Schwarz on the cross terms. If $D \ge S$ the line search is interior and $\Delta_B = S^2/(2D) \ge S/(2(1+\eta(k'-1)))$; if $D < S$ then $\theta = 1$ and $\Delta_B = S - D/2 > S/2$, which is larger. For uncapped pairs $S = \sum_j 2\Delta_j$. □

As $\eta \to 0$ one scan realises the sum of all k' pairwise gains.

Lemma 12 (The certified gap is the FW gap) *At any full scan, with* LB *evaluated at the current $z \ne 0$,* $\text{UB} - \text{LB} = g^{\text{FW}}(z)/\text{UB}$ *(a retained best lower bound gives $\le$), and $g^{\text{FW}}(z) \le 2h + R\sqrt{2h}$ with $R = \max_{u\in Z}\|u\|$, the radius of Z about the origin.*

Proof of Lemma 12 $\text{LB} = \min_{u\in Z}\langle z, u\rangle/\|z\|$ by definition of the supporting hyperplane bound and $\text{UB} = \|z\|$; subtracting, $\text{UB} - \text{LB} = (\langle z, z\rangle - \min_u\langle z, u\rangle)/\|z\| = g^{\text{FW}}/\text{UB}$. At the optimum $0 = \langle z^*, z^*\rangle - \min_u\langle z^*, u\rangle$; subtracting from $g^{\text{FW}}(z)$, $g^{\text{FW}}(z) = (\|z\|^2 - \|z^*\|^2) +$

$(\min_u \langle z^*, u\rangle - \min_u \langle z, u\rangle) \le 2h + \max_u \langle z^* - z, u\rangle \le 2h + R\|z - z^*\| \le 2h + R\sqrt{2h}$ by 1-strong convexity. □

Corollary 13 (Work bound in standard form) *Let $\zeta_k = k+1$ under the one-sided and $2k+1$ under the two-sided schedule. For any $\varepsilon_h > 0$, $h_T \le \varepsilon_h$ as soon as*

$$T \;\ge\; \ell_0 + 16\,\zeta_k\,(M/\delta)^2 \log(h_0/\varepsilon_h),$$

and if $\delta^* > 0$ the certified relative gap $(\mathrm{UB} - \mathrm{LB})/\mathrm{UB}$, evaluated at a full scan with a fresh LB *(Lemma 12), is at most ε whenever $h_t \le \varepsilon^2\delta^{*4}/(8R^2)$. Each iteration costs $O(n+m)$ for the scan, the pair selection and the certificate, $O(k'(n+m) + k'^2)$ for the block step (score update from up to $2k'$ cached columns; Gram matrix of the pair directions from $4k'^2$ cached entries), and $O((n+m)d)$ for every Gram column, or one kernel column, computed on first use, at most one per new receiver or donor and hence at most $n+m$ over the run. The total work to $h_T \le \varepsilon_h$ is therefore*

$$O\Big(\zeta_k (M/\delta)^2 \log(h_0/\varepsilon_h)\,\big(k(n+m) + k^2\big) + N_{\mathrm{cols}}(n+m)d\Big), \qquad N_{\mathrm{cols}} \le n+m.$$

Proof Theorem 6 gives $h_T \le h_0(1-\rho)^{(T-\ell_0)/\zeta_k}$, and $\log(1/(1-\rho)) \ge \rho = (\delta/M)^2/16$ turns $h_0(1-\rho)^{(T-\ell_0)/\zeta_k} \le \varepsilon_h$ into the stated T. By Lemma 12 with $\mathrm{UB} \ge \delta^*$, $(\mathrm{UB} - \mathrm{LB})/\mathrm{UB} = g^{\mathrm{FW}}/\mathrm{UB}^2 \le (2h + R\sqrt{2h})/\delta^{*2} \le 2R\sqrt{2h}/\delta^{*2}$, using $2h \le R\sqrt{2h}$ for $h \le R^2/2$, which every feasible iterate satisfies; this is at most ε iff $h \le \varepsilon^2\delta^{*4}/(8R^2)$. The per-iteration costs are those of Section 4, with $k' \le k$. □

Section 6 refines the drop count in the exponent and replaces the O-constants by fitted timing coefficients.

Theorem 14 (Screening activation) *Assume $\delta^* > 0$. Let $F_V = \{i : \langle z^*, v_i\rangle = \min_j \langle z^*, v_j\rangle\}$ and, with the max, F_W be the optimal faces: the vertices of zero score margin, which contain the optimal supports (possibly strictly). Let $\tau_i = \langle z^*, v_i\rangle - \min_j \langle z^*, v_j\rangle$ be the score margin of v_i (and symmetrically on W), with $\tau > 0$ the minimal nonzero margin. Finally, let L be the larger class diameter.*

A zero-weight point with margin τ_i is screened as soon as $\varrho < \tau_i/(3L)$. Consequently, once $h_t < H := \tau^4\delta^{*2}/(2592\,R^4L^4)$, every zero-weight point of V outside F_V and of W outside F_W is removed permanently at the next screening round. Under Theorem 6 this holds for every iteration $t > \ell_0 + \frac{\zeta_k}{\rho}\log\max\{1, h_0/H\}$, with $\zeta_k = k+1$ one-sided and $2k+1$ two-sided. From the first screening round after this iteration, each scan costs $O(|F_V| + |F_W| + |\operatorname{supp}\alpha| + |\operatorname{supp}\beta|)$ instead of $O(n+m)$, the supports being those of the activating round (a point dropped later stays in the working set).

In words: since $\varrho^2 \le 2R(\mathrm{UB} - \mathrm{LB})$, a point with margin τ_i is screened once the absolute certified gap $\mathrm{UB} - \mathrm{LB}$ falls below $\tau_i^2/(18RL^2)$.

Proof of Theorem 14 Strong convexity gives $\|z - z^*\| \le \varrho$ for the screening radius $\varrho = \sqrt{\mathrm{UB}^2 - \max(\mathrm{LB}, 0)^2}$ (τ is defined only if some point has positive margin; otherwise there is nothing to screen). Let $\vartheta_V = \min_j \langle z^*, v_j\rangle$, choose $j' \in F_V$ and set $v' = v_{j'}$; since $v_{\min}$ minimises $\langle z, \cdot\rangle$, $\langle z^*, v_{\min}\rangle - \vartheta_V = \langle z, v_{\min} - v'\rangle + \langle z^* - z, v_{\min} - v'\rangle \le \varrho L$. Hence

$\langle z, v_i - v_{\min}\rangle = \langle z^*, v_i - v_{\min}\rangle + \langle z - z^*, v_i - v_{\min}\rangle \geq \tau_i - \varrho L - \varrho\|v_i - v_{\min}\|$, and the rule fires when this exceeds $\varrho\|v_i - v_{\min}\|$, for which $\tau_i > 3\varrho L$ suffices since $\|v_i - v_{\min}\| \leq L$. For the activation time, $\varrho^2 \leq 2R(\mathrm{UB} - \mathrm{LB})$ (for $\mathrm{LB} \geq 0$, $\varrho^2 = (\mathrm{UB} - \mathrm{LB})(\mathrm{UB} + \mathrm{LB})$ and $\mathrm{UB} + \mathrm{LB} \leq 2R$; for $\mathrm{LB} < 0$, $\varrho^2 = \mathrm{UB}^2$ and $\mathrm{UB} \leq 2(\mathrm{UB} - \mathrm{LB})$), and $2R(\mathrm{UB} - \mathrm{LB}) \leq 2Rg^{\mathrm{FW}}/\mathrm{UB} \leq (2R/\delta^*)(2h + R\sqrt{2h}) \leq (4R^2/\delta^*)\sqrt{2h}$ (Lemma 12, with equality in the first step for the freshly evaluated bound; the last step uses $h \leq R^2/2$, which holds for every feasible iterate since $h \leq \frac{1}{2}\|z\|^2$); requiring $\varrho^2 \leq \tau^2/(9L^2)$ yields $h_t < H$, and Theorem 6 with $\log(1/(1-\rho)) \geq \rho$ converts it into the iteration bound (the $\max\{1, \cdot\}$ covers $h_0 < H$, where the condition holds from the start by monotonicity). The removal is executed at the first screening round after the condition holds; permanence follows from the safety of the rule. □

Lemmas 3, 12 and 8, Proposition 11 and Theorem 14 are verified numerically on random and adversarial ensembles, at machine precision or to 10^{-9} (Section 8.4; Theorem 14 against faces computed by the SLSQP reference).

6 When does aggregation pay: analysis

Fewer iterations are not the same as less work. This section and Section 9 separate the two. Here: an exact expression for how much more progress a block makes than the single MDM step at the same iterate, a cost model for what the block costs, and a theorem that bounds the drop steps by an identification phase rather than by the crude count of Theorem 6. Section 9 measures every quantity in the model on the five real cells of Section 8.

6.1 Progress per unit of work at one iterate

Consider a case-(a) iteration with k' retained pairs, all uncapped, so that $\gamma_j = \mathrm{gap}_j/\|d_j\|^2$ and pair j alone would gain $\Delta_j = \mathrm{gap}_j^2/(2\|d_j\|^2)$, with Δ_1 the MDM gain. Write

$$Q = \frac{\sum_{j \leq k'} \Delta_j}{\Delta_1}, \qquad \chi = \frac{D}{S}, \qquad S = \sum_j \gamma_j \mathrm{gap}_j = 2\sum_j \Delta_j, \qquad D = \Big\| \sum_j \gamma_j d_j \Big\|^2.$$

Q measures how much useful progress the block has beyond its leading pair and χ how much the directions lose (or gain) by being combined; orthogonal directions give $\chi = 1$. Call the block η-*diverse* if $|\langle d_i, d_j\rangle| \leq \eta\|d_i\|\|d_j\|$ for $i \neq j$ and β_{flat}-*flat*, $0 < \beta_{\mathrm{flat}} \leq 1$, if $\Delta_j \geq \beta_{\mathrm{flat}}\Delta_1$ for every j (the subscript keeps the flatness parameter apart from the W-side weights β).

Proposition 15 (Exact gain ratio) *At such an iteration the exact line search gives*

$$\frac{\Delta_B}{\Delta_1} = Q \cdot \begin{cases} 2 - \chi, & \chi \leq 1, \\ 1/\chi, & \chi \geq 1. \end{cases}$$

If the block is η-diverse then $\chi \leq 1 + \eta(k' - 1)$; if it is β_{flat}-flat then $Q \geq 1 + \beta_{\mathrm{flat}}(k' - 1)$; hence the guarded step gains at least

$$\max(\Delta_B, \Delta_1) \;\geq\; \Delta_1 \, \frac{1 + \beta_{\mathrm{flat}}(k' - 1)}{1 + \eta(k' - 1)}.$$

Proof For uncapped pairs $\gamma_j\|d_j\|^2 = \mathrm{gap}_j$, so $S = \sum_j \mathrm{gap}_j^2/\|d_j\|^2 = 2\sum_j \Delta_j$ and $\varphi_j^2 = \gamma_j^2\|d_j\|^2 = \gamma_j \mathrm{gap}_j$, whence $\sum_j \varphi_j^2 = S$. The line search takes $\theta = \min(S/D, 1)$. If $D \geq S$ the optimum is interior and $\Delta_B = S^2/(2D) = (S/2)/\chi = \sum_j \Delta_j/\chi$; if $D < S$ then $\theta = 1$ and $\Delta_B = S - D/2 = (S/2)(2-\chi)$. Dividing by Δ_1 gives the two cases. For the bound on χ, $D = \sum_{i,j} \gamma_i\gamma_j\langle d_i, d_j\rangle \leq \sum_j \varphi_j^2 + \eta \sum_{i\neq j} \varphi_i\varphi_j \leq (1+\eta(k'-1)) \sum_j \varphi_j^2 = (1+\eta(k'-1))S$ by Cauchy–Schwarz on the cross terms; flatness gives $\sum_j \Delta_j \geq \Delta_1(1+\beta_{\mathrm{flat}}(k'-1))$. When $\chi \leq 1$ the ratio is $Q(2-\chi) \geq Q$, and when $\chi \geq 1$ it is Q/χ; both are at least the stated bound. □

Proposition 15 recovers the diversity bound of Proposition 11 and adds the comparison with Δ_1. The two numbers Q and χ are computed by the guard at every block step (from S, D and the per-pair gaps), so both are observable at no extra cost. Note the direction of the inequalities: the theorem's η bounds the *maximum* absolute pairwise cosine of the retained directions, whereas the *effective correlation*

$$\hat{\eta} := \frac{\chi - 1}{k' - 1} \qquad (k' > 1)$$

that a block step reveals is a γ-weighted aggregate in which cross terms can cancel; the proof gives $\hat{\eta} \leq \eta$, and $\hat{\eta}$ can be negative. Likewise Q/k' close to 1 says the average pair is as good as the leading one, not that every pair is (β_{flat}-flatness). The empirical model of Section 9 is therefore stated in Q and χ, the quantities the guard measures, and the theorem's assumptions (all retained pairs uncapped, maximum cosine at most η, minimum relative gain at least β_{flat}) are recorded separately.

Cost.

Per iteration the solver pays a scan and certificate cost that does not depend on k, a step cost that grows with the number of retained pairs (the k'-column score update and the $O(k'^2)$ guard), and one Gram or kernel column for every receiver or donor not in the cache. Write

$$\mathcal{C}_B = a + b\,k' + c_{\mathrm{col}}\, n_B^{\mathrm{new}}, \qquad \mathcal{C}_1 = a + b + c_{\mathrm{col}}\, n_1^{\mathrm{new}},$$

for the block and the single step from the same state, where $n_B^{\mathrm{new}} \geq n_1^{\mathrm{new}}$ count the column misses of the two candidates ($n_1^{\mathrm{new}} \leq 1$: only the MDM receiver can be new) and a, b, c_{col} are timing coefficients. They are fitted, not assumed: they depend on n, d, the representation (dense, kernel) and the hardware, and Section 9 fits them per cell. The model is a timing approximation of the implementation, not an upper bound on computational work. The guarded block improves progress per unit of work at this iterate exactly when $\max(\Delta_B, \Delta_1)/\Delta_1 > \mathcal{C}_B/\mathcal{C}_1$; by Proposition 15 a sufficient condition is

$$\frac{1 + \beta_{\mathrm{flat}}(k'-1)}{1 + \eta(k'-1)} \;>\; \frac{a + bk' + c_{\mathrm{col}} n_B^{\mathrm{new}}}{a + b + c_{\mathrm{col}} n_1^{\mathrm{new}}}. \tag{2}$$

Two regimes are visible in (2). If c_{col} dominates a and b and $n_1^{\mathrm{new}} > 0$, the right-hand side tends to the miss ratio $n_B^{\mathrm{new}}/n_1^{\mathrm{new}} \leq k'$ (if the MDM receiver is cached and a block receiver is not, it grows with c_{col} and the block cannot pay): the block must earn every new column it touches beyond the single step's, and wins by the full gain

ratio when both need the same column ($n_B^{\text{new}} = n_1^{\text{new}}$), so column dominance raises the hurdle on the iterations on which the block's extra receivers are not cached; it does not exclude a gain. If the per-iteration terms dominate, the right-hand side is $(a + bk')/(a + b)$, linear in k', while the left-hand side saturates at β_{flat}/η; the ratio of the two is maximised at

$$k^* = \sqrt{\frac{a(1-\eta)}{b\,\eta}} \qquad (\beta_{\text{flat}} = 1), \tag{3}$$

the block size at which the marginal cost of one more pair equals its marginal gain. Equation (3) is the unconstrained continuous optimiser of the model for $a, b > 0$ and $0 < \eta < 1$: as $\eta \to 0$ it grows without bound (every pair helps), as $\eta \to 1$ it tends to 0 (only the leading pair helps), and an implementation rounds it and clips it to $\{1, \ldots, k_{\max}\}$.

6.2 Drops are an identification cost

Theorem 6 charges up to k drop steps per non-drop iteration, which would cancel a k-fold gain in progress. The measurements below show that drops happen mostly while the support is being built and become rare once it has settled (on gisette, ijcnn1 and w8a). The following theorem proves this under the standard non-degeneracy assumptions of active-set identification (Bomze et al., 2020; Garber, 2020): after an explicit iteration, every step that removes a non-face point is a drop, no non-face point enters, and once the supports lie in the optimal faces and the weights are close to the optimal ones, no step is a drop at all.

Write $\mathcal{F}_V = \operatorname{conv}\{v_i : i \in F_V\}$ and $\mathcal{F}_W = \operatorname{conv}\{w_j : j \in F_W\}$ for the geometric faces (F_V, F_W remain index sets), and let $V_{F_V} \in \mathbb{R}^{d \times |F_V|}$, $W_{F_W} \in \mathbb{R}^{d \times |F_W|}$ be the matrices whose columns are the face points. A witness pair (p^*, q^*) with $p^* = \sum_i \alpha_i^* v_i$, $q^* = \sum_j \beta_j^* w_j$ and $p^* - q^* = z^*$ carries optimal weights α^*, β^*.

Assumption 16 (A1) *Strict complementarity*: $F_V = \operatorname{supp} \alpha^*$ and $F_W = \operatorname{supp} \beta^*$, and $\alpha_{\min} := \min\big(\min_{i \in F_V} \alpha_i^*,\ \min_{j \in F_W} \beta_j^*\big) > 0$ is the smallest positive optimal weight over both classes. (A2) *Non-degeneracy*: the witness pair (p^*, q^*) is unique, the vertices of each optimal face are affinely independent (so α^*, β^* are unique), and $\sigma > 0$ denotes a constant with $\|\alpha - \alpha^*\|_1 + \|\beta - \beta^*\|_1 \le \|(p - q) - z^*\|/\sigma$ for all $p \in \mathcal{F}_V$, $q \in \mathcal{F}_W$ with face weights α, β (such a constant exists under the two preceding conditions and is computed below). Let $d_{\min} > 0$ be the smallest distance from a face vertex to the affine hull of the other vertices of its face, the minimum taken over the faces with at least two points; if both faces are singletons, (p^*, q^*) is the unique pair of face points and the no-drop conclusions below hold as soon as the supports lie in the faces.

Under (A1) and (A2), affine independence and uniqueness of the witness pair make the linear map $(\alpha - \alpha^*, \beta - \beta^*) \mapsto (p - q) - z^*$ injective on the subspace $U = \{\sum_i (\alpha_i - \alpha_i^*) = 0 = \sum_j (\beta_j - \beta_j^*)\}$ of $\mathbb{R}^{|F_V| + |F_W|}$; if $\sigma_2 > 0$ is the smallest singular value of $[V_{F_V}, -W_{F_W}]$ restricted to U, then $\sigma = \sigma_2/\sqrt{|F_V| + |F_W|}$ works, the square

root being the conversion from the Euclidean to the ℓ_1 norm of the weights. When $U = \{0\}$ (both faces singletons) any $\sigma > 0$ works and the weight condition is void.

The distance to the optimum is controlled by the objective error directly: z^* minimises the 1-strongly convex F over the convex Z, so $\langle z^*, z - z^* \rangle \geq 0$ for every $z \in Z$ and

$$\tfrac{1}{2}\|z - z^*\|^2 \;\leq\; \langle z^*, z - z^* \rangle + \tfrac{1}{2}\|z - z^*\|^2 \;\leq\; h(z), \qquad \text{i.e.} \quad \|z_t - z^*\| \leq \sqrt{2h_t}. \quad (4)$$

(The screening radius $\varrho_t \geq \|z_t - z^*\|$ is the observable proxy for this distance; the theorem does not need it.) Let ℓ_t be the total support size, $\zeta_k = k + 1$ (one-sided) or $2k + 1$ (two-sided schedule), and define

$$H_{\mathrm{id}} = \min\Big\{ \frac{\tau^2}{72L^2},\ \frac{\tau}{12},\ h_3 \Big\}, \qquad h_3 = \max\{h :\ 2h + R\sqrt{2h} \leq \tau/6\},$$
$$H_{\mathrm{w}} = \min\Big\{ H_{\mathrm{id}},\ \frac{(\sigma\alpha_{\min})^2}{8},\ \frac{(\alpha_{\min} d_{\min})^2}{8} \Big\}.$$

Theorem 17 (Identification and drop count) *Let $\delta^* > 0$, let τ, L, R be as in Theorem 14 and let the guarded block-transfer algorithm run with any $k \geq 1$ under either schedule of Theorem 6. Let t_{id} be the first iteration with $h_t < H_{\mathrm{id}}$ and N the number of non-face support indices at t_{id}.*

(i) For every $t \geq t_{\mathrm{id}}$: if either side's support contains a non-face point, the step taken is a drop of a non-face point; and no non-face point enters either support. Hence $\operatorname{supp}\alpha \subset F_V$ and $\operatorname{supp}\beta \subset F_W$ from iteration $t_{\mathrm{id}} + N$ on, and $N \leq \ell_{t_{\mathrm{id}}} \leq \ell_0 + \zeta_k\, t_{\mathrm{id}}$.

(ii) Under Assumption 16, let t_{w} be the first iteration with $h_t < H_{\mathrm{w}}$. No iteration $t \geq \max(t_{\mathrm{w}}, t_{\mathrm{id}} + N)$ is a drop, and no block pair is capped at such an iteration. Consequently every drop occurs before $t_{\mathrm{w}} + N$, the number of drops is at most $\zeta_k(t_{\mathrm{w}} + N) + \ell_0$, and $t_{\mathrm{w}} \leq \ell_0 + 1 + \frac{\zeta_k}{\rho} \log \max\{1, h_0/H_{\mathrm{w}}\}$.

(iii) For $t \geq \max(t_{\mathrm{w}}, t_{\mathrm{id}} + N)$ every iteration contracts. Let $\rho_t^{\mathrm{act}} := 1 - h_{t+1}/h_t$ (for $h_t > 0$) be the actual contraction and, for $j \geq 1$,

$$\underline{\rho}_j := \min\Big\{1,\ 4\rho \max\Big[1, \frac{1 + \beta_{\mathrm{flat}}(j-1)}{1 + \eta(j-1)}\Big]\Big\},$$

which is nondecreasing in j. Then $\rho_t^{\mathrm{act}} \geq \rho$, and $\rho_t^{\mathrm{act}} \geq \underline{\rho}_{k'}$ when the iteration is case (a) with an η-diverse, β_{flat}-flat block of k' pairs (under the two-sided schedule, the block of the first, larger-gap side, which is the step the progress argument uses).

Proof Throughout, $s_i = \langle g, v_i \rangle = -\langle z, v_i \rangle$ on side V and $s_j = \langle z, w_j \rangle$ on side W; we argue on side V, side W being symmetric with the maximum in place of the minimum.

Three facts. Write $\varpi_t := \|z_t - z^*\| \leq \sqrt{2h_t}$ by (4). (F1) If $\varpi \leq \tau/(6L)$ then for $i \notin F_V$ and $j \in F_V$, $s_j - s_i = \langle z, v_i - v_j \rangle = \langle z^*, v_i - v_j \rangle + \langle z - z^*, v_i - v_j \rangle \geq \tau_i - \varpi L \geq 5\tau/6$, while for $i, j \in F_V$, $|s_i - s_j| \leq \varpi L \leq \tau/6$; in particular the global argmax of s is a face point. (F2) Let ω_t be the total weight on non-face points of both sides. By convexity $h_t \geq \langle \nabla F(z^*), z_t - z^* \rangle =$

$\langle z^*, z_t - z^*\rangle$; writing $\vartheta_V := \min_i \langle z^*, v_i\rangle$ and $\vartheta_W := \max_j \langle z^*, w_j\rangle$, so that $\langle z^*, v_i\rangle = \vartheta_V + \tau_i$ and $\langle z^*, w_j\rangle = \vartheta_W - \tau_j$, and using $\|z^*\|^2 = \langle z^*, p^* - q^*\rangle = \vartheta_V - \vartheta_W$ (the optimal weights live on the faces, where the margins vanish), $\langle z^*, z_t - z^*\rangle = \sum_i \alpha_i\tau_i + \sum_j \beta_j\tau_j \geq \tau\,\omega_t$. Hence $\omega_t \leq h_t/\tau$. (F3) By (4) and (F2), $h_t < H_{\mathrm{id}}$ implies $\varpi_t \leq \tau/(6L)$, $\omega_t \leq \tau/(72L^2)$, $\omega_t \leq 1/12$ and $2h_t + R\sqrt{2h_t} \leq \tau/6$.

(i) Identification. Let $h_t < H_{\mathrm{id}}$ and suppose $\operatorname{supp}\alpha \not\subset F_V$. The worst active donor u minimises s over the support, so by (F1) $u \notin F_V$. The away gap of side V is $g_V^{\mathrm{A}} = \sum_i \alpha_i(s_i - s_u) \geq \sum_{i\in F_V} \alpha_i \cdot 5\tau/6 \geq (1-\omega_t)5\tau/6 \geq 55\tau/72$. A side whose support lies in its face has $g^{\mathrm{PW}} \leq g^{\mathrm{A}} + g^{\mathrm{FW}} \leq \varpi L + (2h + R\sqrt{2h}) \leq \tau/3$ by (F1) and (F3), so the side with the larger pairwise gap, which the schedule steps first, is a side with a non-face support point whenever one exists. On that side pair 1 is (r^*, u) with $r^* \in F_V$ the global argmax, so $\mathrm{gap}_1 \geq 5\tau/6$ and the uncapped transfer would be $\mathrm{gap}_1/\|d_1\|^2 \geq 5\tau/(6L^2) > \tau/(72L^2) \geq \omega_t \geq \alpha_u$: pair 1 is capped and the iteration is case (b). The away step on u has derivative $\langle g, p - v_u\rangle = g_V^{\mathrm{A}} \geq 55\tau/72$ and curvature $\|p - v_u\|^2 \leq L^2$, so its unconstrained length is at least $55\tau/(72L^2)$, while its maximal length is $\alpha_u/(1-\alpha_u) \leq \frac{12}{11}\alpha_u \leq \tau/(11L^2)$; the away step is boundary-clipped, i.e. a drop of u, and it adds no index. If instead neither side has a non-face support point, all donors are face points and a non-face receiver r has $\mathrm{gap} = s_r - s_u \leq -5\tau/6 < 0$ against any of them, so it is discarded (the discard rule of Algorithm 1); the toward candidate is $r^* \in F_V$. Under the two-sided schedule the second side steps only on non-drop iterations, when neither side has a non-face point, and the same argument applies. Thus for $t \geq t_{\mathrm{id}}$ (monotonicity of h) the set of non-face support indices never grows and loses one member at every iteration while it is nonempty, which proves (i); the bound on N is the support count of Theorem 6.

(ii) No late drops. Let $t \geq \max(t_{\mathrm{w}}, t_{\mathrm{id}} + N)$, so $p_t \in \mathcal{F}_V$, $q_t \in \mathcal{F}_W$ and, by (4) applied to the strict inequality $h_t < H_{\mathrm{w}}$, $\varpi_t < \min\{\tau/(6L), \sigma\alpha_{\min}/2, \alpha_{\min}d_{\min}/2\}$. By (A2), $\|\alpha_t - \alpha^*\|_\infty \leq \varpi_t/\sigma < \alpha_{\min}/2$, so by (A1) every face index carries weight $\alpha_{t,i} > \alpha_i^* - \alpha_{\min}/2 \geq \alpha_{\min}/2 > 0$: the supports equal the faces and every support weight exceeds $\alpha_{\min}/2$. Any pair (r, u) of face points has $\mathrm{gap}_j = s_r - s_u = \langle z - z^*, v_u - v_r\rangle \leq \varpi_t\|d_j\|$, because $\langle z^*, \cdot\rangle$ is constant on the face, and $\|d_j\| = \|v_r - v_u\| \geq d_{\min}$ (the distance between two vertices is at least the distance from one of them to the affine hull of the others), so its uncapped transfer satisfies

$$\gamma_j \;=\; \frac{\mathrm{gap}_j}{\|d_j\|^2} \;\leq\; \frac{\varpi_t}{\|d_j\|} \;\leq\; \frac{\varpi_t}{d_{\min}} \;<\; \frac{\alpha_{\min}}{2} \;<\; \alpha_{u_j}\,:$$

no pair is capped, the iteration is case (a), no away step is taken, and a block transfer removes strictly less than α_{u_j} from each donor, so no index leaves the support. (If both faces are singletons there are no pairs and nothing to show.) Drops at $t \geq t_{\mathrm{w}}$ are therefore among the N identification drops of (i), which occur at the consecutive iterations $t_{\mathrm{id}}, \ldots, t_{\mathrm{id}} + N - 1$; hence all drops occur before $t_{\mathrm{w}} + N$, and the count of Theorem 6 over the first $t_{\mathrm{w}} + N$ iterations gives the bound. The bound on t_{w} is Theorem 6 with $\log(1/(1-\rho)) \geq \rho$.

(iii) Rate. A non-drop iteration gains at least the amount of Lemma 4, which with Lemma 5 is ρh_t. In case (a) the guard takes $\max(\Delta_B, \Delta_1)$, which is at least Δ_1 (the guard) and, by Proposition 15, at least $\Delta_1(1 + \beta_{\mathrm{flat}}(k' - 1))/(1 + \eta(k' - 1))$; hence at least $\Delta_1 \max[1, (1 + \beta_{\mathrm{flat}}(k' - 1))/(1 + \eta(k' - 1))]$. In case (a) the proof of Lemma 4 gives $\Delta_1 \geq (g^{\mathrm{PW}})^2/(8M^2) \geq 2\delta^2 h_t/(8M^2) = 4\rho h_t$ by Lemma 5. A gain cannot exceed h_t, whence the minimum with 1. Monotonicity of $\underline{\rho}_j$ in j: the derivative of $(1+\beta_{\mathrm{flat}}(j-1))/(1+\eta(j-1))$ in j has the sign of $\beta_{\mathrm{flat}} - \eta$, so the ratio is nondecreasing when $\beta_{\mathrm{flat}} \geq \eta$, and when $\beta_{\mathrm{flat}} < \eta$ it lies below 1 and the outer maximum makes $\underline{\rho}_j$ constant. □

Theorem 17 says that drops are paid during identification and never afterwards: the worst case k drops per iteration of Theorem 6 applies only up to $t_{\mathrm{w}} + N$, and the

k-dependence of the drop count enters only through the support that is built up to then. The measurements in Figure 4 are consistent with this shape on gisette, ijcnn1 and w8a. The thresholds are those of active-set identification and are pessimistic (Section 9.4 measures by how much); the measured drop counts below are 0.3–70% of one class's final support.

Corollary 18 (Conditional contraction and iteration count) *Run the algorithm with requested block size k. At iteration t let $k'_t \in \{1,\dots,k\}$ be the number of pairs retained by the block used in the progress argument (the first side's block; $k'_t = 1$ on case-(b) and drop iterations, where only the leading pair is used). Let $t_1(k) := \max(t_{\mathrm{w}}, t_{\mathrm{id}} + N)$ as in Theorem 17 and suppose every case-(a) iteration $t \ge t_1(k)$ is η-diverse and β_{flat}-flat. Then for every $T \ge t_1(k)$,*

$$h_T \;\le\; h_{t_1(k)} \prod_{t=t_1(k)}^{T-1} \bigl(1 - \rho_t^{\mathrm{act}}\bigr), \qquad \rho_t^{\mathrm{act}} \;\ge\; \underline{\rho}_{k'_t}$$

on case-(a) iterations (after $t_1(k)$ every iteration is case (a) with no capped pair, by Theorem 17(ii), so the convention $k'_t = 1$ matters only before $t_1(k)$). In particular, for a target ε_h on the objective error (distinct from the relative certificate tolerance ε of Section 3; Corollary 13 converts one into the other): if $h_{t_1(k)} > \varepsilon_h$ and $k'_t \ge \underline{k}$ for some $\underline{k} \in \{1,\dots,k\}$ on every case-(a) iteration after $t_1(k)$, then $h_T \le \varepsilon_h$ for $T = t_1(k) + \bigl\lceil \log(h_{t_1(k)}/\varepsilon_h)/\underline{\rho}_{\underline{k}} \bigr\rceil$ when $\underline{\rho}_{\underline{k}} < 1$ (and $T = t_1(k) + 1$ when $\underline{\rho}_{\underline{k}} = 1$); if $h_{t_1(k)} \le \varepsilon_h$ the run is over at $t_1(k)$.

Proof Theorem 17(iii) with k' read as k'_t gives the product; since $\underline{\rho}_j$ is nondecreasing in j, $\underline{\rho}_{k'_t} \ge \underline{\rho}_{\underline{k}}$, and $\log(1/(1-\underline{\rho})) \ge \underline{\rho}$ turns the product into the iteration count. □

Timing model.

The corollary counts iterations; what they cost is a matter of the fitted coefficients, not of a theorem. Let $k_t^{\mathrm{ev}} \le 2k$ be the total number of pairs the implementation assembled and evaluated at iteration t (both sides' blocks under the two-sided schedule, including the pairs evaluated before an away drop) and $N_{\mathrm{cols}}(k)$ the number of distinct columns computed over the whole run. The cost model approximates the run's work by

$$\begin{aligned} W(k) \;&\approx\; c_{\mathrm{col}} N_{\mathrm{cols}}(k) + \sum_{t<T} \bigl(a + b\, k_t^{\mathrm{ev}}\bigr) \\ &\lesssim\; c_{\mathrm{col}} N_{\mathrm{cols}}(k) + (a + 2bk)\Bigl[t_1(k) + \Bigl\lceil \frac{\log(h_{t_1(k)}/\varepsilon_h)}{\underline{\rho}_{\underline{k}}} \Bigr\rceil\Bigr], \end{aligned}$$

the second expression using the iteration count of Corollary 18 and $k_t^{\mathrm{ev}} \le 2k$. Here b is a cost per evaluated pair; the b fitted in Section 9 is a cost per iteration and requested size k, which under the two-sided schedule absorbs the factor two. Corollary and model together describe one run. They do not by themselves compare two block sizes: $N_{\mathrm{cols}}(k)$, $t_1(k)$ and $h_{t_1(k)}$ all depend on the trajectory, and comparing the multiplicative factors of two upper bounds is not a proof that one algorithm is faster. What they do say is which factor aggregation can act on. The per-iteration term after

identification carries the factor $(a+bk_t^{\mathrm{ev}})/\underline{\rho}_{k'_t}$, and if $k'_t \equiv k_t^{\mathrm{ev}} \equiv k$ (one-sided schedule, full blocks) and $\beta_{\mathrm{flat}} = 1$ this is proportional to $(1+\eta(k-1))(a+bk)/k$, minimised at k^* of (3); the step rule does not reduce the column term, though a block can raise it by bringing in extra receivers. Whether N_{cols}, t_1 and h_{t_1} are in fact insensitive to k is an empirical question, answered below for the five cells (column counts within 20% across k; drops and misses concentrated in the same early deciles for every k). A comparative speed-up theorem would need those three quantities bounded uniformly in k, which we do not prove.

7 Exact reductions of the SVM variants

L_2 soft margin.

The problem $\min \frac{1}{2}\|w\|^2 + \frac{C}{2}\sum_i \xi_i^2$ subject to $y_i(\langle w, x_i\rangle + b) \geq 1 - \xi_i$ is the hard-margin problem on the augmented points $\tilde{x}_i = [x_i;\, e_i/\sqrt{C}]$, e_i the i-th standard basis vector, whose Gram matrix is $K + I/C$ (K the Gram matrix of the original points) and whose augmented hulls never intersect; let δ_C denote their distance. The augmentation is never materialised: the extra coordinates of an iterate are its convex weights scaled by $1/\sqrt{C}$, so every augmented inner product is a cached original-space product plus a sparse diagonal correction, $\langle \tilde{p}, \tilde{v}_i\rangle = \langle p, v_i\rangle + \alpha_i/C$ and $\|\tilde{p}\|^2 = \|p\|^2 + \|\alpha\|^2/C$. The separator is recovered as $w = 2(p-q)/\delta_C^2$, $b = (\|\tilde{q}\|^2 - \|\tilde{p}\|^2)/\delta_C^2$, and the primal value is $P = 2/\delta_C^2$. Validation: the solver agrees with the SLSQP (sequential least-squares quadratic programming) reference on the explicitly augmented problem to 10^{-10} across random instances and values of C; the regularisation path δ_C is monotone in C; on a9a (C=0.1) its certified primal 687.07 equals LIBLINEAR's to five digits.

ν-SVM.

The L_1 (hinge) soft margin is the nearest-point problem between *reduced* convex hulls (Bennett & Bredensteiner, 2000) $\mathcal{R}(V,\mu) = \{\sum a_i v_i : \sum a_i = 1,\ 0 \leq a_i \leq \mu\}$, $\mu = 2/(\nu(n+m))$ (Crisp & Burges, 2000; Mavroforakis & Theodoridis, 2006). Two changes: the extreme-point oracle becomes a capped top-k blend (weight μ on the $\lfloor 1/\mu \rfloor$ best points in the search direction and the remainder $1 - \mu\lfloor 1/\mu \rfloor$ on the next, via a partial sort, still $O(n)$), and the primary step is the capped MDM transfer with blended toward steps as fallback; caching and the reduced support-function lower bound carry over unchanged. Validation: agreement with the exact box-constrained quadratic program (QP) to 10^{-9}–10^{-14} and with LIBSVM's NuSVC to $\sim 10^{-10}$ through the mapping $\delta^* = \|w_{\mathrm{svc}}\|/\Lambda$, w_{svc} LIBSVM's weight vector and Λ the per-class sum of the rescaled dual coefficients. The reduced hull is a polytope, so (1) still applies, but its vertices are blends and the capped transfer is not the pairwise step analysed in Section 5; the analysis of this variant is left open.

Kernels.

The algorithm touches data only through inner products, so kernelisation changes the data access, not the algorithm: Gram columns become kernel columns $\kappa(V, x_i)$, squared norms become kernel diagonals, iterates exist purely as convex weights (the

exact cache refresh is reconstructed from the support's cached kernel columns), and the classifier is the kernel expansion of the witness-pair bisector. A ridge $K+I/C$ gives the kernel L_2 margin by the same diagonal trick. Validation: the linear kernel reproduces the Euclidean solver to 10^{-12}; radial-basis-function (RBF) distances match the exact kernel QP to 10^{-9} and LIBSVM's hard-margin solution, via $\|w_{\mathcal{H}}\|^2 = (\alpha \circ y)^\top K(\alpha \circ y)$, $w_{\mathcal{H}}$ the feature-space normal, for LIBSVM's dual coefficients α and labels y, to 10^{-5}; kernel SMO matches the QP to 10^{-8}.

8 Experiments

Setup.

All solvers are validated against a reference QP solution (SLSQP from SciPy on the weight variables, function tolerance 10^{-14}, on instances small enough for it; distances agree to 10^{-9} or better). Stopping rule $\varepsilon = 10^{-3}$ on the certified relative gap unless stated; baselines use matched tolerances on the same objective where they expose one, LIBLINEAR up to its intercept penalty (Section 8.3). Synthetic instances follow the protocol of Gupta and Kalantari (2026) as implemented in the released code (two Gaussian clouds, $n = 5{,}000$ per class). Real data are the LIBSVM sets a9a, w8a, ijcnn1, gisette and covtype (100k subsample), standardised, official test sets where they exist, five seeds, 600 s per solver and cell, on two dedicated AMD EPYC machines (Section 8.3 lists which ran what). LIBSVM (Chang & Lin, 2011) and LIBLINEAR (Fan et al., 2008) are called through scikit-learn (Pedregosa et al., 2011); "our SMO" is our own implementation of sequential minimal optimisation (Platt, 1999) and K-ETA the kernelised ETA.

Blocks and screening.

The block-size and step ablations on real data are the measurements of Section 9 (Table 12); the batteries below use the untuned k=16, which the September instrumented runs of Section 9 put within 16% of the best size on every cell. On the synthetic protocol screening gives 1.3–2.4× at $\varepsilon = 10^{-5}$ and 0.9–1.3× at 10^{-3} (Figure 2); Section 8.1 reports it on a real cell.

Real data, regime battery (Table 2).

TA I first decides separability to its tolerance (Table 6): on gisette it returns a witness pair in 9–12 s; on the other four it returns an iterate with $\|p-q\| \le 10^{-3}$, an ε-witness that the hulls are within 10^{-3} of touching, in 0.1–1 s except on w8a, where it takes 24,867 iterations and 28–30 s. Any solver with a FW-gap certificate can produce the same bound; the value here is that it is computed before the distance run and decides which of the baselines is meaningful. On gisette (d=5,000; 1,237 of 6,000 points in the support) ETA ties LIBSVM on the linear hard margin (38.2 ± 3.2 vs 38.5 ± 3.1 s), and our full-cache SMO has a 1.1× lower mean (overlapping intervals); K-ETA is 1.2× faster than LIBSVM on the RBF hard margin (64.6 ± 6.2 vs 75.6 ± 3.1 s, separated intervals), with identical solutions. On the kernel L_2 margins K-ETA matches kernel SMO's solution, and since both now cache every column or row they compute, it computes as many kernel columns as SMO computes rows. Kernel SMO ties it on

Table 2 Regime battery on real data, five seeds, 600 s per solver, mean ± 95% confidence interval (CI; for w8a KHM, whose seeds differ by orders of magnitude, median [maximum]). Kernel cells (RBF kernel $\exp(-\|x - x'\|^2/d)$) use a class-balanced subsample of n_k points. Same objective per row: LIN, linear hard margin (ETA with screening vs LIBSVM's C=10^6 surrogate; our SMO in brackets); KHM, feature-space hard margin (K-ETA vs the same surrogate); KL2, kernel L_2 margin, C=1 (K-ETA vs kernel SMO, exact objective; in brackets LIBSVM on the precomputed $K + I/C$ kernel, fit time + kernel construction, from the September run on the earlier machine, Section 8.3). conv = seeds in which ETA certified a distance or an intersection; oracle = $O(nd)$ column (ETA) or row (SMO) evaluations, thousands; both solvers cache every column or row they compute. Released code on the AMD EPYC 9754 machine, four single-threaded cells at a time (Section 8.3).

cell	dataset	n_k / d	ETA (s)	baseline (s)	conv	dist. ETA / base	acc. ETA / base	oracle
LIN	gisette	6000 / 5000	38.2 ± 3.2	38.5 ± 3.1 [34.0 ± 3.2]	5/5	3.6638 / 3.6640	.981 / .981	1.5 [1.4]
KHM	gisette	6000 / 5000	64.6 ± 6.2	75.6 ± 3.1	5/5	.05215 / .05215	.979 / .979	2.6
KHM	ijcnn1	9853 / 22	242 ± 69	1.1 ± 0.2	5/5	.00430 / .00430	.946 / .946	2.6
KHM	a9a	10000 / 123	> 600	18.5 ± 2.0	0/5	$\leq$.00002 / .0033	.685 / .752	6.0
KHM	covtype	10000 / 54	> 600	122 ± 33 (maxiter)	0/5	$\leq$.00044 / .00025	.711 / .712	7.6
KHM	w8a	6479 / 300	0.0 [36.2]	—	5/5	hulls intersect	—	0.5
KL2	gisette	6000 / 5000	89.8 ± 7.8	90.6 ± 8.5 [$0.5 \pm 0.1 + 3$]	5/5	.06882 / .06882	.971 / .971	3.7 / 3.6
KL2	w8a	6479 / 300	6.3 ± 1.0	5.2 ± 1.5 [$0.6 \pm 0.1 + 1$]	5/5	.05682 / .05682	.9745 / .9745	3.6 / 3.6
KL2	a9a	10000 / 123	13.2 ± 1.0	8.9 ± 1.5 [$2.2 \pm 0.2 + 2$]	5/5	.03019 / .03019	.795 / .795	7.6 / 7.6
KL2	covtype	10000 / 54	16.8 ± 4.3	5.8 ± 1.2 [$2.1 \pm 0.1 + 2$]	5/5	.02609 / .02609	.792 / .792	9.1 / 9.1
KL2	ijcnn1	9853 / 22	5.3 ± 0.3	2.6 ± 0.3 [$1.1 \pm 0.1 + 2$]	5/5	.04790 / .04790	.959 / .959	4.6 / 4.5

gisette and w8a and is 1.5–2.9× faster on a9a, covtype and ijcnn1. In the September run, with a 512-row SMO cache, K-ETA computed 4–6× fewer columns than SMO recomputed rows and was up to 3.3× faster: that advantage was the baseline's cache policy, not the step rule. Against BPCG (Tsuji et al., 2022) with the same column cache and certificate (Table 13), block ETA needs 12–21× fewer iterations, is 4.4–7.2× faster on the four kernel L_2 cells of the isolated runs (35× and 2.5× on gisette KL2 in a loaded September run), and ties on the gisette L_2 margin, where columns dominate. LIBSVM on the precomputed $K + I/C$ kernel (brackets in Table 2), run only in September on the earlier machine, reached the same distances 2.7–22× faster than K-ETA did in that run (K-ETA then took 7.7–85 s), kernel construction included; K-ETA's kernel L_2 results therefore beat only our BPCG implementation.

The KHM column shows where the geometry turns against the method. On ijcnn1 both return the same solution (819 support vectors), but K-ETA needs $1.5–2.7 \times 10^5$ iterations and 242 s against LIBSVM's 1.1 s: the hulls are 0.004 apart in a space of diameter $\approx \sqrt{2}$, so a relative certificate needs an absolute gap of order 10^{-6} (a log-factor cost; δ/M is translation-invariant and unknown here), which plausibly suits LIBSVM's second-order selection better. On a9a and covtype the hulls are within 3×10^{-5} and 6×10^{-4} of touching (largest seed): K-ETA times out with an uncertified iterate and LIBSVM's C=10^6 surrogate stops at its Karush–Kuhn–Tucker (KKT) tolerance (iteration cap on covtype), so neither certifies anything and the LIBSVM entries of those two rows are not solutions of the hard margin either. On w8a the hulls intersect (duplicate points, opposite labels); K-ETA detects this (an ε-witness) in under 0.1 s in four seeds and in 36 s in the fifth.

Real data, L_2 soft margins (Table 7).

The same boundary appears within one problem class. Gisette is separable, so its L_2 margin has a sparse support and ETA converges to the certified gap in 19–21 s at every C; our full-cache SMO ties (1.1–1.3× lower means, overlapping intervals). LIBLINEAR's two solvers behave very differently here (Table 10). Its primal Newton solver, which the batteries ran, needs 4 min to 1.2 h at the matched tolerance 10^{-6} and stops 3–24% above the certified optimum at its default 10^{-4}; its dual coordinate-descent solver reaches the certified optimum in 3–4.5 s at every tolerance from 10^{-4} to 10^{-6}, 2.9–4.5× faster than ETA's isolated 13.0 s at C=1 on the same machine (Table 12; the battery's 19–21 s come from the other machine, under load). On gisette ETA therefore adds only the certificate. On the four overlapping datasets LIBLINEAR's Newton solver finishes in 0.1–24 s (its dual solver is far slower there: 131 s to 31 min on a9a). Within the cap ETA converges only on a9a and ijcnn1 at C=0.1 (583 and 336 s, one a9a seed just over the cap), where our SMO takes 163 and 59 s; our SMO also converges on ijcnn1 at C=1 and w8a at C=0.1. At C=10 the capped ETA iterate loses 1–27 accuracy points, and on covtype it is not usable at any C (Table 7). The uncapped traces (Figure 1) show a rate at least linear, improving along the run (the plotted certified gap scales like $\sqrt{h}$, Corollary 13), and reach the certified 10^{-3} in 259–792 s, 30–1,800× LIBLINEAR's matched-tolerance time on those three cells (the traces ran on the EPYC 7B13 and the LIBLINEAR battery on the EPYC 9754; on a9a, where the sweep gives a same-machine time, the ratio is 280×); the nine capped cells give

only lower bounds on that ratio. Heuristically (δ has no known bound in terms of the class hulls), if a fixed fraction of the points is active and the n_s active atoms form a near-regular simplex, its pyramidal width is $\sqrt{2/n_s}$ times the side (Lacoste-Julien & Jaggi, 2015), so $\rho \sim 1/n_s$ and the cost would be $O(n^2)$-type; the n-scaling experiment of Section 8.2 measures time growing as $n^{2.1}$–$n^{2.7}$, with the hull distance also shrinking in n.

8.1 Screening on a real cell

Theorem 14 is exercised on gisette, the one separable real dataset, in the linear hard margin (LIN) and the L_2 margin (C=1), at $\varepsilon = 10^{-3}$ and 10^{-5} with screening off and on (every 25 iterations, compaction when at least 5% of a class is removable), three seeds, one configuration at a time in a fresh process (`src/screening_real.py`). Reported per configuration (Table 3): time, iterations, columns, the number of points alive at the end of 6,000, the number of compactions and the time of the first, and the distance, which must agree with the unscreened run.

The rule does activate on real data, and it does not pay here. At $\varepsilon = 10^{-3}$ no round removes 5% of a class on either cell; the runs are identical with and without screening. At 10^{-5} the certificate closes enough in the last 2 s of a 29 s run for 15–16 compactions to remove 70% of the points (1,738 and 1,808 survive of 6,000, against optimal supports of 1,237 and 1,238), with the distance and the iteration count unchanged. Wall-clock is a tie within the confidence intervals, as the September isolated runs of the same configurations predict: there the scan that screening shortens is under 1% of the gisette time and the columns 88–93% of it, and the screened points never had columns. Screening buys time only where the scan is a large share of the iteration and the certificate closes early in the run, the regime of Figure 2; on the real cells of this paper the scan never is.

Frequency and cost. A screening round is one $O(n + m)$ pass over the cached scores plus at most one Gram or kernel column per side (that of $v_{\min}$ or $w_{\max}$, cached afterwards). At one round every 25 iterations the overhead is below timing noise: at $\varepsilon = 10^{-3}$ the screened and unscreened runs of Table 3 take identical trajectories and tie in time. The frequency and the 5% compaction threshold were not tuned, and sensitivity to them was not measured. The sufficient condition of Theorem 14, $\varrho < \tau_i/(3L)$, keeps holding once it holds, because UB is nonincreasing and the retained LB nondecreasing, so a round every 25 iterations removes such a point at most 25 iterations later than a round at every iteration would (the 5% compaction threshold can delay it further). On the overlapping datasets screening was not run: their L_2 supports are dense and the certificate does not close within the cap on most cells, so there would be little to remove. The wall-clock attributable to screening on real data is therefore measured only on gisette, where it is a tie.

8.2 Scaling with n on overlapping L_2 instances

The heuristic $\rho \sim 1/n_s$ of Section 8 predicts iterations growing linearly and time quadratically in n when the support fraction is constant. Table 4 measures both on synthetic L_2 overlap instances ($d \in \{100, 1000\}$, C=1, n points per class, guarded

Table 3 Safe screening on gisette (linear hard margin and L_2 margin at C=1; 6,000 points, d=5,000), three seeds, one process at a time (`src/screening_real.py`). Screening every 25 iterations, compaction when at least 5% of a class is removable. Alive = points remaining at the end; first = time of the first compaction; the optimal support has 1,237 (LIN) and 1,238 (L_2) points. Iterations, columns and distance are identical with and without screening on every seed.

cell	ε	screening	time (s)	iterations	columns	alive	compactions	first (s)
LIN	10^{-3}	off	27.3 ± 8.6	821	1,468	6,000	0	—
LIN	10^{-3}	on	26.9 ± 6.8	821	1,468	6,000	0	—
LIN	10^{-5}	off	28.7 ± 6.2	1,331	1,469	6,000	0	—
LIN	10^{-5}	on	29.0 ± 7.6	1,331	1,469	1,738	16	27.0
L_2	10^{-3}	off	28.4 ± 6.4	831	1,480	6,000	0	—
L_2	10^{-3}	on	26.9 ± 7.2	831	1,480	6,000	0	—
L_2	10^{-5}	off	29.0 ± 6.3	1,361	1,482	6,000	0	—
L_2	10^{-5}	on	27.1 ± 0.6	1,361	1,482	1,808	15	25.3

blocks k=16, $\varepsilon = 10^{-3}$, one process at a time; `src/n_scaling.py`). Time grows as $n^{2.1}$ at d=100 and $n^{2.7}$ at d=1,000 over the range (log–log slopes), so the $O(n^2)$-type label is a lower estimate, and the growth is not the one the heuristic gives. The support fraction is not constant: it doubles at d=100 and at d=1,000 falls from 0.31 to 0.05 and back to 0.10 as n passes d, while the iteration count grows as $n^{1.5}$ and $n^{2.6}$, far faster than the support in both cases; the last doubling at d=1,000 multiplies the iterations by 24. What does move with n on both families is the hull distance itself, from 0.26 to 0.036 at d=100 and from 2.0 to 0.042 at d=1,000: at fixed C more points overlap more, the augmented hulls approach each other, and on these instances the pyramidal width of the difference atom set presumably shrinks with them ($\rho \propto (\delta/M)^2$; δ is translation-invariant and δ^* is not, so this is an observation about the instances, not an inequality). The near-regular-simplex picture accounts for the support term only and under-predicts the exponent.

8.3 Experimental details

Solver settings and tolerance equivalence.

ETA: block transfers with $k = 16$, guarded, away fallback, stop at certified relative gap 10^{-3}, iteration cap 2×10^6 and 600 s cap in the batteries (the n-scaling instances have a 3,600 s cap, the screening runs 3,600 s, the synthetic tables 200,000 iterations and no time cap). Implementation details that the analysis does not model: the certificate and the stopping test are evaluated every fifth iteration, on the other four the receiver is searched over the points touched so far (`prioritized=True`, `full_scan_every=5`), the cached scores are recomputed exactly every 100 iterations (this refresh is most of the "rest" column of Table 12), a side whose best score is within 10^{-11} of its worst active score is skipped before the side comparison, and the adaptive size starts from $k = 16$. Every table and figure comes from the released code, and the block algorithm (schedule of Lemma 8, case-(b) fallback in every path) is the same throughout, except that Figure 2 and the comparison configuration of Figure 3 use the development-phase

Table 4 Scaling with n (points per class) on synthetic L_2 overlap instances, C=1, guarded blocks k=16, $\varepsilon = 10^{-3}$, one process at a time. Support = points with positive weight at the solution (fraction of $2n$); the "slope" rows are the log–log slopes against n over the rows above them.

d	n	time (s)	iterations	columns	support (frac.)	distance	drops
100	1,000	0.6	716	229	130 (.07)	0.258	13
100	2,000	2.7	2,736	741	387 (.10)	0.138	75
100	5,000	18.8	10,701	2,404	1,133 (.11)	0.078	599
100	10,000	84.5	28,261	5,294	2,623 (.13)	0.052	1,595
100	20,000	326.5	62,741	8,883	5,209 (.13)	0.036	2,557
	slope	2.13	1.49	1.22	1.22		
1,000	1,000	0.7	281	704	610 (.31)	2.031	23
1,000	2,000	2.2	706	957	750 (.19)	1.434	89
1,000	5,000	14.1	3,001	1,348	884 (.09)	0.578	180
1,000	10,000	131.7	32,551	1,932	1,085 (.05)	0.100	350
1,000	20,000	2,812	781,011	6,895	3,906 (.10)	0.042	782
	slope	2.71	2.57	0.68	0.53		

toward-step solver and the synthetic step-ablation counts come from the development log. The guarantees of Section 5 are for the full-scan algorithm; the prioritised search, the certificate every fifth iteration and the 5% compaction threshold of screening are heuristics outside the analysis, under which every step still decreases the objective (an exact line search along a direction with positive gap). The October results (Tables 2, 6, 7, 10, 11, 12 and 13 and Figures 1, 2 and 3) were timed with the final code; of the September results named in the hardware paragraph below, groups (i) and (ii) were timed before two constant-factor changes (vectorised donor selection, block Gram matrix from cached entries), with trajectories unchanged, and group (iii) after them. `src/rerun_new_machine.sh` reproduces the October results in one sequence; `src/provenance_rerun.sh`, `src/block_diagnostics.py`, `src/adaptive_sweep.py` and `src/libsvm_kl2.py` the September ones. Screening runs every 25 iterations in the LIN cell only, where it never activated at 10^{-3} (Theorem 14 is a sufficient condition, which these runs stop before meeting); Section 8.1 exercises it at 10^{-5}. Our SMO: maximal-violating-pair selection, KKT gap 10^{-3}, same caps; hard margin as $C = 10^{12}$; kernel SMO on $K + I/C$; in the October runs a cache that keeps every row it computes, as the ETA, BPCG and LIBSVM caches do (the September runs, and covtype in Table 7, used a 512-row least-recently-used cache, so their row counts include re-computations; in the synthetic benchmark, where SMO is not plotted, it ran with a two-row cache). LIBSVM on the precomputed $K + I/C$ kernel (scikit-learn `SVC`, `kernel='precomputed'`, C=10^6, tolerance 10^{-3}, five seeds; `src/libsvm_kl2.py`) solves the same kernel L_2 margin: the RBF kernel with $1/C$ on the diagonal is built with the Basic Linear Algebra Subprograms (BLAS) in 1–3 s and the fit takes 0.5–2.2 s, with distances equal to K-ETA's to five digits. LIBLINEAR (scikit-learn 1.8 `LinearSVC`, which with `dual='auto'` selects LIBLINEAR's primal trust-region Newton solver on every dataset here, since $n \geq d$; the iteration counts below are Newton iterations): squared hinge with parameter $C/2$, so that its objective equals $\frac{1}{2}\|w\|^2 + \frac{C}{2}\sum \xi_i^2$; tolerance 10^{-6}, chosen so that its final primal matches ETA's certified value on every cell where ETA converges except gisette at C=10 (ijcnn1 550.61 vs

550.61; gisette 0.14900 vs 0.14893, inside ETA's certified 10^{-3}; a9a 687.07 vs 687.07); this tolerance is also why the Newton solver needs 17,550 iterations and 32 minutes on gisette at C=1, and 44,569 iterations and 1.2 h at C=10, where it nevertheless stops 4.3% above the certified optimum (0.15532 vs 0.14899). The gisette LIBLINEAR times in Table 7 are therefore the cost of matching ETA's primal with the Newton solver. Table 10 sweeps the tolerance for both solvers: at scikit-learn's default 10^{-4} the Newton solver takes 3–40 s on gisette but stops 3–24% above the certified optimum with lower test accuracy, while the dual solver reaches the optimum in 3–4.5 s at every tolerance; on a9a the order reverses. Intercept scaling 100, so LIBLINEAR regularises $b/100$ where the geometric formulation leaves b free (the primals agree to five digits); iteration cap 2×10^5. LIBSVM (scikit-learn `SVC`): tolerance 10^{-3} (an absolute KKT criterion), 1000 MB cache, iteration cap 2×10^6 (reported as *maxiter* when hit) and no wall-clock cap; hard margins as $C = 10^6$; RBF kernel $\exp(-\|x - x'\|^2/d)$. Neither LIBSVM nor LIBLINEAR exposes the certified relative gap, so what justifies calling their cells "the same solution" is the agreement of the distances to four or five digits, not a matched stopping rule. Oracle counts: ETA counts Gram or kernel columns evaluated against a full class ($O(nd)$ or $O(n_k d)$ each); SMO counts kernel rows, the same unit. Columns are cached without eviction and the block Gram matrix Γ is assembled from four cached entries per pair, so peak cache memory is the column count times n_k doubles: 0.73 GB for the largest cell (covtype KL2, 9,100 columns), 0.61 GB for a9a KL2 and 0.36 GB for ijcnn1 KL2; measured peak resident memory in the isolated runs of Table 12 is 0.74 GB on gisette (data-dominated), and on the kernel cells 0.49–0.93 GB for ETA and 0.67–1.29 GB for BPCG (per-cell means).

Data and seeds.

Table 5 lists the sets. a9a, w8a, ijcnn1 and gisette use the LIBSVM training and test files; covtype (binary, scaled) is split 75/25 by seed and its training part subsampled to 100,000 points. Features are standardised on the training set. Kernel cells use a class-balanced subsample of at most 5,000 points per class (w8a's minority class has 1,479 points, hence n_k = 6,479; gisette uses all 6,000). Seeds vary the covtype split, the kernel subsamples and the timing environment; on datasets with fixed splits the deterministic solvers return identical solutions across seeds, which is why accuracies are reported without intervals. Confidence intervals are t-based 95% intervals over seeds; on the capped cells of Table 7 the accuracy of the returned iterate varies across seeds (a9a C=10: $.594 \pm .230$), which the table's single accuracy figure hides. gisette's test set has 1,000 points, so accuracy differences of 0.003–0.008 there are 3–8 examples, about two binomial standard errors; w8a's majority-class rate is 0.970 and ijcnn1's 0.905, so their accuracies near those values are weak evidence of anything. Large language models (Claude, Anthropic) were used to draft code, experiment scripts and text; every result was run and checked by the author.

Hardware and protocol.

The October rerun used two dedicated cloud VMs with the same software (Linux, Python 3.14.7, NumPy 2.4.4 with OpenBLAS 0.3.31, SciPy 1.17.1, scikit-learn 1.8.0, `OMP_NUM_THREADS=1`, no other load). A 16-vCPU AMD EPYC 9754 with 60 GB RAM

Table 5 Datasets (LIBSVM binary sets, features standardised on the training part). covtype is the class-2-versus-rest binary set, split 75/25 by seed with the training part subsampled to 100,000; kernel cells use a class-balanced subsample of the training part (Section 8.3).

dataset	$n + m$ (train)	test	d	positive fraction	nonzeros
gisette	6,000	1,000	5,000	0.500	0.99
a9a	32,561	16,281	123	0.241	1.00
w8a	49,749	14,951	300	0.030	1.00
ijcnn1	49,990	91,701	22	0.097	1.00
covtype	100,000	145,253	54	0.488	0.98

Table 6 TA I on the five datasets (linear kernel, full training sets, five seeds, absolute tolerance 10^{-3} on $\|p - q\|$): "separated" returns a witness pair with a positive certified lower bound; "intersect" returns an iterate with $\|p - q\| \le 10^{-3}$, an ε-witness that the hulls are within 10^{-3} of touching, not a proof of intersection. Iterations and the final $\|p - q\|$ are seed-0 values (covtype: mean over its five splits).

dataset	verdict	iterations	time (s)	final $\|p - q\|$
gisette	separated	235	10.3 ± 1.1	5.07
a9a	intersect (ε-witness)	381	0.7 ± 0.0	9.9×10^{-4}
w8a	intersect (ε-witness)	24,867	28.9 ± 1.1	1.0×10^{-3}
ijcnn1	intersect (ε-witness)	83	0.1 ± 0.0	9.5×10^{-4}
covtype	intersect (ε-witness)	221	0.6 ± 0.3	9.8×10^{-4}

(`results/machine_oct2026.txt`) ran the regime battery (Table 2) and the synthetic benchmark (Figure 3) four single-threaded cells at a time, the L_2 battery (Table 7) up to four at a time, and the runs of Figure 2 one at a time. A 32-vCPU AMD EPYC 7B13 with 128 GB RAM (`results/machine_oct2026_covtype.txt`) ran the step ablation and the BPCG comparison (Tables 12 and 13), one configuration at a time in a fresh process; the LIBLINEAR sweep (Table 10), three processes at a time; the uncapped traces (Figure 1), one at a time; and covtype's LIBLINEAR entries in Table 7. Comparisons are made within one machine except: the trace-to-LIBLINEAR ratios of Section 8 and five of the six rows of Table 11; covtype's rows of Table 7 (September ETA and SMO beside October LIBLINEAR); the brackets of Table 2 (September LIBSVM times beside October K-ETA times); and the dual-solver comparison of Section 8, which sets ETA in isolation against LIBLINEAR run three processes at a time. Cross-machine ratios can be off by more than 1.5×: ETA on the gisette L_2 margin at C=1 took 13.0 s in isolation on the EPYC 7B13 and 20.6 s in the EPYC 9754 battery, a gap that mixes the processors with the battery's load. Three groups of results come from September. (i) covtype in Table 7: a full SMO row cache on its 100,000 training points needs about 80 GB, more than the EPYC 9754 has, and a run on the EPYC 7B13 was also killed for lack of memory. Its ETA and SMO entries are therefore the September run, on a 32-vCPU Intel Xeon 2.6 GHz VM (122 GB RAM, Python 3.12, four cells at a time, SMO with a 512-row cache), where

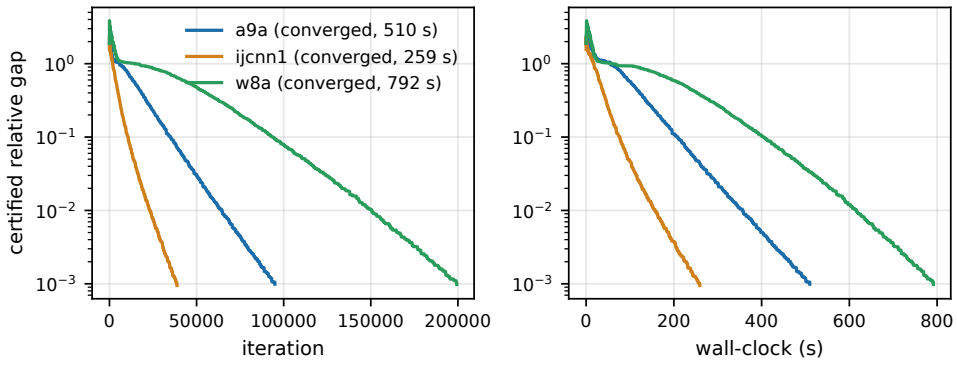


Fig. 1 Certified relative gap of ETA on a9a, w8a and ijcnn1 (L_2, $C = 0.1$, uncapped, seed 0), versus iteration and wall-clock: linear-rate convergence in the dense-support regime, reaching the certified 10^{-3} in 259–792 s against the 600 s cap of Table 7 (AMD EPYC 7B13, one run at a time).

both timed out on every cell. (ii) The LIBSVM-precomputed brackets of Table 2 and the gisette KL2 row of Table 13 are September runs on the Intel machine, made concurrently with up to 19 single-threaded processes. (iii) Tables 14, 15, 16, 3 and 4 and the synthetic tables of this section were not rerun: they are September runs with the final code (the adaptive sweep on a local development machine, the others on cloud VMs), made one process at a time or instrumented, and are read only for ratios within each table. Five seeds unless stated, t-based 95% intervals over seeds. Concurrent load can manufacture ratios: under September's 19-process load guarded k=1 appeared 1.3× faster than MDM on gisette, a tie in isolation, and the gisette column time ranged from 28 to 36 s across three isolated runs of the same trajectory (`results/step_profile{,2,3}.json`); only ratios whose intervals separate, and exceed that variation, are read as results.

Reference solver in the KL2 cells.

The hinge-loss `SVC(rbf, C=1)`, a different objective, took 4.5 ± 0.4 (a9a), 3.1 ± 0.3 (covtype), 75.1 ± 4.8 (gisette), 0.9 ± 0.1 (ijcnn1) and 2.2 ± 0.7 s (w8a) in the run of Table 2, with test accuracies 0.789, 0.783, 0.977, 0.950 and 0.975, within one point of the L_2 solutions and on either side of them.

Step ablation, blocks and shrinking (synthetic).

Iteration counts at $\varepsilon = 10^{-6}$ on single instances from the development log (`src/zigzag_bench.py`; not part of the rerun driver): d=30, n=200: toward 158,941, midpoint 148,996, away 106, pairwise 181; d=100, n=1,000: toward and midpoint fail within 300,000, away 15,886, pairwise 3,921. At the working tolerance 10^{-3} the pairwise/away gain over the midpoint heuristic of Gupta and Kalantari (2026) is about 11% on those instances. Block transfers cut iterations 3–25× relative to single MDM (e.g. 43,486 → 3,001 at k=16 on a dense-support L_2 instance at d=1,000, Table 9) and are the only configuration that reaches the certified tolerance on MNIST odd-vs-even within the iteration cap of that benchmark. Figure 2 shows the screening speed-ups

Table 7 L_2 soft-margin battery: five seeds, 600 s per solver for ETA and our SMO (our own implementation; LIBSVM would need an $n \times n$ precomputed kernel), LIBLINEAR (primal Newton solver, Section 8.3) at tolerance 10^{-6} with no time cap; released code with a full SMO row cache, up to four single-threaded cells at a time on the AMD EPYC 9754 machine, except covtype (below). Mean ± 95% CI, which on the fixed-split datasets (all but covtype) measures timing noise only (w8a, C=0.1: one LIBLINEAR seed took 71 s, the other four 12–14 s). "cap" = stopped at the time or iteration cap in every seed; * four of five seeds converged within the cap, the mean counting the capped seed at 600 s. ETA's certified relative gap is reported at its stopping point (mean [min, max]); accuracies are those of the returned iterate. covtype: ETA and SMO from the September run on the earlier machine with SMO's 512-row cache, because a full cache needs about 80 GB (Section 8.3); LIBLINEAR on an AMD EPYC 7B13, whose objective values match the September run's to seven or more significant digits.

dataset ($n + m$ / d)	C	ETA (s)	SMO (s)	LIBLINEAR (s)	ETA gap at stop	acc. ETA / SMO / LIBLINEAR
gisette (6,000 / 5,000)	0.1	19.3 ± 2.4	16.7 ± 2.6	261 ± 4	9×10^{-4}	.981 / .981 / .981
	1	20.6 ± 3.5	15.9 ± 3.3	$1{,}893 \pm 41$	10×10^{-4}	.981 / .981 / .981
	10	20.9 ± 1.4	19.6 ± 1.1	$4{,}380 \pm 277$	10×10^{-4}	.981 / .981 / .978
a9a (32,561 / 123)	0.1	$583 \pm 17^*$	163 ± 4	1.9 ± 0.1	$(9.9\text{–}11) \times 10^{-4}$	.849 / .849 / .849
	1	cap	cap	3.2 ± 0.8	0.66 [0.55, 0.85]	.840 / .850 / .849
	10	cap	cap	1.1 ± 0.1	1.2 [1.2, 1.2]	.594 / .786 / .849
ijcnn1 (49,990 / 22)	0.1	336 ± 61	59.0 ± 7.4	0.1 ± 0.0	9×10^{-4}	.918 / .918 / .918
	1	cap	334 ± 11	0.2 ± 0.1	0.17 [0.16, 0.18]	.920 / .918 / .918
	10	cap	cap	0.2 ± 0.0	1.1 [0.99, 1.2]	.649 / .917 / .918
w8a (49,749 / 300)	0.1	cap	470 ± 50	24.4 ± 32.2	0.044 [0.016, 0.12]	.987 / .987 / .987
	1	cap	cap	6.8 ± 1.6	0.86 [0.84, 0.89]	.979 / .987 / .987
	10	cap	cap	8.5 ± 1.5	1.4 [1.4, 1.5]	.975 / .977 / .987
covtype (100,000 / 54)	0.1	cap	cap	12.8 ± 11.1	1.0 [1.0, 1.0]	.572 / .701 / .756
	1	cap	cap	3.2 ± 1.5	1.1 [1.0, 1.1]	.537 / .545 / .756
	10	cap	cap	2.5 ± 0.9	1.3 [1.3, 1.6]	.502 / .515 / .756

and Figure 3 the consolidated comparison (`src/final_benchmark.py`), five trials per class (mean ± 95% CI). Its "optimised" configuration is k=16 with screening on the hard-margin classes, k=32 on the L_2 class and k=8 on MNIST (the 5,000-image subset distributed with the mlxtend package, odd versus even digits); the comparison configuration is the toward-step algorithm of Gupta and Kalantari (2026) on the hard-margin rows, single MDM on the L_2 and MNIST rows and pairwise K-ETA on RBF, with a 2×10^5-iteration cap; our SMO there runs with a 300 s cap and a two-row cache. On the four hard-margin classes the optimised solver ranges from 1.5× faster than LIBSVM to 1.1× slower (0.47 ± 0.06 vs 0.69 ± 0.15 s at d=1,000, the only separated intervals; 14.1 ± 1.5 vs 13.2 ± 1.9 s at d=10,000; 0.59 ± 0.11 vs 0.63 ± 0.15 s at ε=10^{-5}, where the toward-step configuration fails to converge in every trial; RBF 1.64 ± 0.42 vs 1.94 ± 0.34 s); on the synthetic L_2 margin (d=1,000, C=1) it ties LIBLINEAR (13.7 ± 11.0 vs 14.5 ± 5.0 s); on MNIST odd-vs-even it is 2.1× slower (135 ± 17 vs 63 ± 13 s; single MDM does not converge within its cap) and on the ν-SVM 1.4× slower (41.8 ± 7.9 vs NuSVC's 30.9 ± 5.7 s). Distances agree within tolerance on every row. On the four classes where the comparison configuration also converges, blocks and guard change wall-clock by 0.85–1.2×; their clear wins are the two where the comparison configuration does not converge within the cap (the toward step at $\varepsilon = 10^{-5}$, single MDM on MNIST).

Table 8 Two-sided schedules on the synthetic instances of Section 8 (n=5,000 per class, k=16, $\varepsilon = 10^{-3}$ for the L_2 rows, one process at a time, single instances): the unconditional V-then-W order versus the schedule of Lemma 8 (larger-gap side first, second side skipped after a drop), an ablation of the skip. Drop steps and skipped second-side steps are counted over the run.

instance	schedule	time (s)	iterations	drops	skips	distance
hard margin, d=1000, ε=10^{-5}	unconditional	0.8	16	0	0	55.339365
hard margin, d=1000, ε=10^{-5}	Lemma 8	0.7	16	0	0	55.339365
L_2 overlap, d=100, C=1	unconditional	23.1	12,311	785	0	0.078400
L_2 overlap, d=100, C=1	Lemma 8	21.5	10,701	599	316	0.078400
L_2 overlap, d=1000, C=1	unconditional	14.2	2,901	182	0	0.578143
L_2 overlap, d=1000, C=1	Lemma 8	14.9	3,001	180	121	0.578143

8.4 Numerical verification of the analysis

Lemma 3 was checked on 4,000 random ensembles including heavily clipped and boundary cases (worst slack at machine precision); Proposition 11 on 500 near-orthogonal ensembles; Lemma 12 to 10^{-9} at arbitrary iterates; Lemma 8 on instrumented two-sided runs (no drop iteration added a support index, and every non-drop iteration met the gain bound with M replaced by the computable upper bound $\operatorname{diam} V + \operatorname{diam} W$); and Theorem 14 on instances whose optimal face was computed by the SLSQP reference: at every activating round all zero-weight non-face points were screened, and the final working set lay in the union of the optimal face and the final support. Over

Table 9 Block-size ablation on the same instances (same settings): single MDM (k=1) versus blocks of k transfers. Iterations fall with k; wall-clock need not: blocks touch more columns, and on the d=100 L_2 instance they add receivers (1,345 to 3,076 columns) and single MDM is fastest.

instance	k	time (s)	iterations	columns	drops
hard margin, d=1000, ε=10^{-5}	1	0.7	301	104	0
hard margin, d=1000, ε=10^{-5}	4	0.7	36	103	0
hard margin, d=1000, ε=10^{-5}	16	0.7	16	103	0
hard margin, d=1000, ε=10^{-5}	32	0.7	16	109	0
L_2 overlap, d=100, C=1	1	17.4	72,546	1,345	0
L_2 overlap, d=100, C=1	4	23.1	24,606	1,632	416
L_2 overlap, d=100, C=1	16	21.7	10,701	2,404	599
L_2 overlap, d=100, C=1	32	25.8	7,606	3,076	305
L_2 overlap, d=1000, C=1	1	23.4	43,486	1,242	0
L_2 overlap, d=1000, C=1	4	18.5	10,071	1,330	177
L_2 overlap, d=1000, C=1	16	15.1	3,001	1,348	180
L_2 overlap, d=1000, C=1	32	15.1	1,741	1,396	170

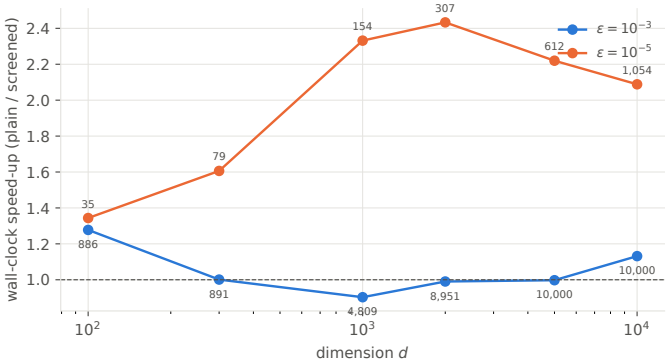


Fig. 2 Safe screening on the separated synthetic protocol (n=10,000, two trials): wall-clock speed-up of screening versus dimension at $\varepsilon = 10^{-3}$ and 10^{-5}; labels are surviving points of 10,000. Made with the toward-step solver of the development phase (`src/shrink_benchmark.py`); the rule and its activation do not depend on the step, and Table 3 gives the released solver on real data.

5,000 instrumented live block steps none increased the objective. All checks are test files in the released code.

Table 10 LIBLINEAR's two solvers across tolerances on gisette and a9a (`src/liblin_sweep.py`; five seeds, three processes at a time and nothing else running, AMD EPYC 7B13). Newton: `LinearSVC` with `dual='auto'`, which selects the primal trust-region Newton solver, the one the batteries ran; dual: `dual=True`, dual coordinate descent. Primal objective relative to the best known value P^* (0.14835 and 0.14893 on gisette at C=0.1 and 1, ETA's certified values; 687.07 and 6,869.7 on a9a); on these fixed splits every seed returns the same primal and accuracy. On a9a at C=1 the dual solver hit its 2×10^5 iteration cap in four of five seeds at 10^{-6} and in all five at 10^{-8}, so those times are lower bounds.

dataset	C	tol	Newton (s)	Newton primal	dual (s)	dual primal	acc. Newton / dual
gisette	0.1	10^{-4}	40.1 ± 0.5	0.15345 (+3.4%)	3.0 ± 0.1	0.14835 (+0.0%)	.978 / .981
		3×10^{-5}	85.7 ± 0.8	0.14922 (+0.6%)	3.3 ± 0.2	0.14835 (+0.0%)	.980 / .981
		10^{-5}	151 ± 2	0.14842 (+0.05%)	3.7 ± 0.6	0.14835 (+0.0%)	.981 / .981
		10^{-6}	269 ± 2	0.14835 (+0.0%)	4.5 ± 0.7	0.14835 (+0.0%)	.981 / .981
gisette	1	10^{-4}	3.0 ± 0.2	0.18440 (+24%)	2.9 ± 0.2	0.14893 (+0.0%)	.974 / .981
		3×10^{-5}	4.0 ± 0.3	0.18343 (+23%)	3.2 ± 0.1	0.14893 (+0.0%)	.974 / .981
		10^{-5}	398 ± 18	0.15568 (+4.5%)	3.5 ± 0.1	0.14893 (+0.0%)	.977 / .981
		10^{-6}	$1{,}813 \pm 67$	0.14900 (+0.05%)	4.5 ± 0.7	0.14893 (+0.0%)	.981 / .981
a9a	0.1	10^{-4}	0.5 ± 0.1	687.08 (+0.001%)	131 ± 38	687.07 (+0.0%)	.849 / .849
		10^{-6}	1.8 ± 0.4	687.07 (+0.0%)	180 ± 44	687.07 (+0.0%)	.849 / .849
		10^{-8}	2.4 ± 0.5	687.07 (+0.0%)	226 ± 50	687.07 (+0.0%)	.849 / .849
a9a	1	10^{-4}	0.6 ± 0.1	6,869.7 (+0.001%)	$1{,}291 \pm 416$	6,869.7 (+0.0%)	.850 / .849
		10^{-6}	2.5 ± 0.3	6,869.7 (+0.0%)	$1{,}828 \pm 271$	6,869.7 (+0.0%)	.849 / .849
		10^{-8}	5.9 ± 0.8	6,869.7 (+0.0%)	$1{,}852 \pm 249$	6,869.7 (+0.0%)	.849 / .849

Table 11 Time to 1% of the optimum, seed-0 traces. The ETA columns measure *dual* progress and LIBLINEAR's column *primal* progress, so this is not a matched primal comparison (the primal of ETA's returned (w, b) is not recorded in the traces). For ETA the certified dual value $2/\mathrm{UB}_t^2$ increases to the optimum, so $s_t = 1 - (2/\mathrm{UB}_t^2)/P^*$ with P^* the best known primal (the lower of LIBLINEAR's best value and ETA's certified value); "certified 1%" is the first time the certificate itself proves $1 - \mathrm{LB}_t^2/\mathrm{UB}_t^2 \le 1\%$, and "cert. 10^{-3}" the time the stopping rule $(\mathrm{UB} - \mathrm{LB})/\mathrm{UB} \le 10^{-3}$ is met. ETA's a9a, w8a and ijcnn1 times are from the uncapped runs of Figure 1 (`src/time_to_tol.py`); its gisette times are from the battery traces of Table 7. Machines: ETA's gisette entries and LIBLINEAR's upper bounds come from the EPYC 9754, ETA's a9a, w8a and ijcnn1 entries and LIBLINEAR's swept times from the EPYC 7B13. LIBLINEAR: the mean time of the coarsest tolerance of Table 10 whose primal is within 1% of P^* in every seed, for the Newton (N) and the dual (D) solver; where the sweep did not run, the matched 10^{-6} Newton run of Table 7, an upper bound (on gisette at C=10 that run stops 4.3% above P^*). On the remaining cells ETA is not within 1% at the 600 s cap ($s = 3$–89%).

dataset	C	P^*	ETA 1%	ETA 0.1%	cert. 1%	cert. 10^{-3}	LIBLINEAR 1% (N / D)
gisette	0.1	0.14835	17 s	18 s	19 s	19 s	86 s / 3.0 s
gisette	1	0.14893	20 s	21 s	21 s	22 s	1,813 s / 2.9 s
gisette	10	0.14899	20 s	21 s	22 s	22 s	none at 10^{-6} / not run
a9a	0.1	687.07	218 s	295 s	402 s	510 s	0.5 s / 131 s
w8a	0.1	108.80	435 s	559 s	670 s	792 s	$\le$ 24 s / not run
ijcnn1	0.1	550.61	80 s	125 s	189 s	259 s	$\le$ 0.14 s / not run

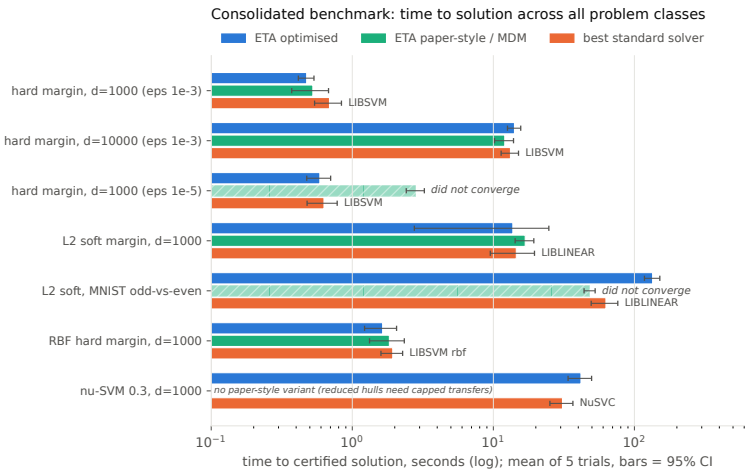


Fig. 3 Time to certified solution on seven problem classes: the optimised configuration vs the toward-step or single-MDM configuration vs the best standard solver per class (settings in Section 8.3); five trials per class (October rerun, AMD EPYC 9754), mean with 95% CI. The ν-SVM row has no toward-step bar because the algorithm of Gupta and Kalantari (2026) has no reduced-hull variant (reduced hulls need capped transfers).

9 When does aggregation pay: measurements

9.1 Step ablation on real data

Table 12 separates the effects the batteries of Section 8 mix: the receiver search (prioritised against exact), the guard and away fallback (guarded k=1 against plain MDM), aggregation (k=16 and the adaptive size of Section 9.3 against guarded k=1), and the comparison with away-step FW and BPCG, all in isolated runs with a time split. The guard and away fallback cut the iterations of MDM with the prioritised search 1.4–2.7× and its time by at most 1.3× (a tie on gisette and ijcnn1); an exact receiver search cuts iterations as much (1.4–2.7×) and, on the four kernel cells, time by 1.8–1.9×, and with exact search the guard changes iterations by under 10% either way, so that gain is the search's, not the guard's; the guard and fallback themselves cost step time (with exact search the guarded single pair is 1.4–1.7× slower than MDM on the kernel cells), the price of the guarantee. Aggregation cuts iterations a further 3.8–7.0×, which buys wall-clock only where per-iteration $O(n)$ work, not column computation, dominates: on the four kernel cells 2.1–2.8× over guarded k=1, 1.2–1.9× over exact-search MDM and 4.4–7.2× over BPCG, and nothing on gisette.

With the prioritised search the guarded single pair halves the iterations of MDM on ijcnn1 (1.4× on gisette), taking drop steps on 0.1–2.4% of its iterations; the exact-search rows show the same cut without a guard. Blocks then cut iterations a further 3.8–7.0× (instrumented seed-0 runs at k=4 are in Table 14: within 32% of k=16 in time on every cell). Away fallbacks that drop an index occur on 0.1–36% of guarded iterations, and their share grows with k: larger blocks add more indices per iteration, and the drops remove them. The time split says where fewer iterations buy time. On gisette, 82–89% of every configuration's time except away-step FW's is Gram-column computation ($O(nd)$ per new receiver, d=5,000), so MDM, guarded k=1, block k=16, the adaptive size and BPCG all lie within 11% of one another at 12–13 s, inside the between-run variation of the column time, so those rows should be read as a tie. Iteration, column and drop counts equal those of the September runs for every configuration both contain except plain MDM, whose prioritised search varies between runs (up to 5% across seeds, under 0.3% between the two runs' means); only the times changed. Peak resident memory is 0.74 GB on gisette and, as per-cell means on the kernel cells, 0.49–0.93 GB for ETA and 0.67–1.29 GB for BPCG. On ijcnn1 the columns cost under 1 s and the rest is per-iteration $O(n)$ work: the score update and line search inside the step, the certificate in the remainder, and least of all the scan itself. That work falls with the iteration count, though not in proportion, because a block step updates the scores with up to k' columns; the net effect on ijcnn1 is 2.1× over MDM, 1.2× over exact-search MDM and 4.4× over BPCG at k=16, and 2.1×, 1.2× and 4.5× with the adaptive size, which settles at k=7–8, the k^* of Table 14; the other three kernel cells behave alike (Table 12). Away-step FW with exact search, the classical alternative to pairwise steps, needs 30–110× more iterations than plain MDM where it converges (gisette and w8a, 207 and 226 s) and times out on the other three kernel cells.

BPCG here is the blended pairwise step of Tsuji et al. (2022) on the nm product atoms $v_i - w_j$ of Z, with product-atom capacities, the separable linear minimisation

Table 12 Step ablation on real data with a time split, isolated runs (`src/step_profile.py`: three seeds, each configuration alone in its own process on an otherwise idle AMD EPYC 7B13, with the final code): the gisette L_2 margin (C=1, d=5,000) and the four kernel L_2 cells of Table 2. MDM is plain pairwise FW (single transfers, no guard, no fallback) with the prioritised receiver search and schedule of the other rows; "-ex." rows replace that search by an exact linear minimisation over all points; AFW is away-step FW; guarded k=1 is Algorithm 1 with a single pair (guard and case-(b) fallback active); auto is the adaptive block size of Section 9.3 (median k over the run given); BPCG as in Table 13. Split of the ETA time: scan = the score scan and pair selection; col = Gram or kernel column computation; step = the step itself (block assembly, guard, line search, cache and score update); rest = certificate, bounds and score refresh. Drops = away fallbacks that removed a support index. Every run converged to the certified gap, distances agreeing to that tolerance and test accuracies to 2×10^{-4} per dataset and seed, except AFW on the ijcnn1, a9a and covtype kernel cells, which timed out at 600 s with certified gaps of 0.0012–0.004. KL2 = kernel L_2.

instance	variant	k	time (s)	scan	col	step	rest	iterations	columns	drops
gisette L_2	MDM	1	12.1 ± 0.2	0.2	10.2	0.8	0.9	4,946	1,300	0
gisette L_2	MDM-ex.	1	12.0 ± 0.2	0.1	10.6	0.7	0.7	3,546	1,341	0
gisette L_2	guarded	1	12.8 ± 0.1	0.2	10.5	1.5	0.6	3,471	1,334	82
gisette L_2	guarded-ex.	1	12.9 ± 0.4	0.1	10.7	1.5	0.7	3,416	1,341	87
gisette L_2	AFW-ex.	1	206.7 ± 5.3	7.6	10.8	90.9	97.3	527,108	1,340	0
gisette L_2	block	16	13.0 ± 0.1	0.1	11.6	1.2	0.2	831	1,480	301
gisette L_2	auto	9	12.7 ± 0.3	0.1	11.3	1.1	0.2	986	1,451	194
gisette L_2	BPCG	—	13.2 ± 0.1					10,470	1,322	87
ijcnn1 KL2	MDM	1	7.8 ± 0.5	1.2	0.8	2.2	3.6	12,763	4,523	0
ijcnn1 KL2	MDM-ex.	1	4.4 ± 0.2	0.2	0.8	1.2	2.3	5,528	4,519	0
ijcnn1 KL2	guarded	1	8.1 ± 0.4	0.8	0.7	4.1	2.6	6,118	4,514	14
ijcnn1 KL2	guarded-ex.	1	7.4 ± 0.5	0.1	0.7	3.9	2.6	6,070	4,520	14
ijcnn1 KL2	AFW-ex.	1	> 600	11.8	0.7	254.1	333.5	658,667	4,504	0
ijcnn1 KL2	block	16	3.8 ± 0.3	0.2	0.6	2.3	0.7	1,396	4,590	98
ijcnn1 KL2	auto	8	3.7 ± 0.1	0.2	0.6	2.0	0.8	1,734	4,570	75
ijcnn1 KL2	BPCG	—	16.6 ± 1.5					18,034	4,512	19
w8a KL2	MDM	1	6.6 ± 0.7	0.9	1.6	1.8	2.3	11,491	3,588	0
w8a KL2	MDM-ex.	1	3.4 ± 0.2	0.1	1.4	0.8	1.1	4,307	3,585	0
w8a KL2	guarded	1	5.2 ± 0.5	0.4	1.4	2.4	1.1	4,243	3,581	8
w8a KL2	guarded-ex.	1	5.0 ± 0.4	0.1	1.4	2.3	1.1	4,248	3,586	11
w8a KL2	AFW-ex.	1	225.7 ± 3.4	5.3	1.5	107.1	111.7	350,050	3,569	0
w8a KL2	block	16	2.5 ± 0.2	0.1	1.3	0.9	0.2	603	3,591	50
w8a KL2	auto	10	2.5 ± 0.1	0.1	1.3	0.9	0.2	756	3,590	40
w8a KL2	BPCG	—	13.4 ± 1.2					12,342	3,577	12
a9a KL2	MDM	1	23.3 ± 0.3	3.1	3.1	5.8	11.3	22,879	7,609	0
a9a KL2	MDM-ex.	1	12.8 ± 0.1	0.3	2.8	3.1	6.6	9,616	7,608	0
a9a KL2	guarded	1	19.5 ± 1.1	1.7	2.7	8.5	6.7	9,879	7,611	27
a9a KL2	guarded-ex.	1	18.0 ± 0.4	0.2	2.6	8.3	6.8	9,852	7,607	22
a9a KL2	AFW-ex.	1	> 600	7.4	3.0	245.6	344.2	412,530	7,543	0
a9a KL2	block	16	6.9 ± 0.6	0.3	2.4	3.0	1.2	1,489	7,616	35
a9a KL2	auto	13	7.0 ± 0.3	0.3	2.5	3.0	1.3	1,641	7,607	26
a9a KL2	BPCG	—	49.1 ± 1.5					28,653	7,605	24
covtype KL2	MDM	1	29.3 ± 1.7	3.9	2.2	7.5	15.7	26,748	9,067	0
covtype KL2	MDM-ex.	1	16.2 ± 0.4	0.3	2.0	4.3	9.6	11,789	9,066	0
covtype KL2	guarded	1	26.2 ± 1.8	2.3	1.8	11.9	10.2	12,544	9,069	12
covtype KL2	guarded-ex.	1	24.6 ± 1.3	0.3	1.9	11.9	10.6	12,602	9,068	14
covtype KL2	AFW-ex.	1	> 600	6.4	2.0	243.8	348.0	354,002	9,026	0
covtype KL2	block	16	12.2 ± 0.2	0.7	1.6	6.7	3.1	3,336	9,074	26
covtype KL2	auto	7	12.4 ± 0.8	0.8	1.6	6.2	3.8	4,154	9,071	19
covtype KL2	BPCG	—	70.0 ± 4.6					38,422	9,065	15

Table 13 BPCG (Tsuji et al., 2022) versus block ETA (k=16) on the same objective with the same certificate (ε=10^{-3}), the same column cache and the same 600 s cap (solver `src/bpcg.py`): the isolated runs of Table 12 (three seeds, AMD EPYC 7B13), and for gisette KL2, which those runs do not include, the September run (`src/bpcg_baseline.py`, three seeds, concurrent with the regime battery on the earlier machine), marked †. Every cell converged; distances agree per cell to five digits and test accuracies to 10^{-4}. Columns are kernel or Gram columns computed; drops count BPCG's capped local steps and ETA's away drops.

cell	solver	time (s)	iterations	columns	drops
gisette L_2, C=1	BPCG	13.2 ± 0.1	10,470	1,322	87
gisette L_2, C=1	ETA	13.0 ± 0.1	831	1,480	301
gisette KL2†	BPCG	213 ± 28	8,702	3,641	3
gisette KL2†	ETA	83.9 ± 19.0	251	3,653	4
w8a KL2	BPCG	13.4 ± 1.2	12,342	3,577	12
w8a KL2	ETA	2.5 ± 0.2	603	3,591	50
a9a KL2	BPCG	49.1 ± 1.5	28,653	7,605	24
a9a KL2	ETA	6.9 ± 0.6	1,489	7,616	35
covtype KL2	BPCG	70.0 ± 4.6	38,422	9,065	15
covtype KL2	ETA	12.2 ± 0.2	3,336	9,074	26
ijcnn1 KL2	BPCG	16.6 ± 1.5	18,034	4,512	19
ijcnn1 KL2	ETA	3.8 ± 0.3	1,396	4,590	98

oracle of Lemma 1, exact line searches and the certificate of Lemma 12; its scores are maintained from the same cached columns, so column counts are comparable, but its active set is a Python-level dictionary over product atoms, and on that parametrisation it needs 1.1–2.1× more iterations than even plain MDM with side transfers (gisette 10,470 vs 4,946; ijcnn1 18,034 vs 12,763). The 12–21× iteration ratio and the 4.4–7.2× wall-clock ratio against block ETA on the four isolated kernel cells (35× and 2.5× on gisette KL2 in September) therefore compare parametrisation and implementation as well as step rules; a BPCG on the side-transfer parametrisation is listed in Section 10. The block's fewer scans buy time where scans dominate (KL2) and nothing where columns do (gisette L_2).

9.2 Per-step measurements

Every quantity in Proposition 15, the cost model and Theorem 17 was recorded on the five real cells of Tables 7 and 2 (gisette L_2 and the kernel L_2 cells of ijcnn1, w8a, a9a and covtype, seed 0; `src/block_diagnostics.py`, in September, the twenty configurations one after the other in one process on an otherwise idle cloud VM, $k \in \{1, 4, 16, 32\}$). At every block step the solver logs Δ_B, Δ_1, Q, χ, the candidate taken, the capped pairs, the column misses, the support size and a time split of the step next to a dry-run timing of the single MDM update from the same state. Table 14 fits the model per cell: $\hat{\eta} = (\chi - 1)/(k' - 1)$ (median over the block calls with pair 1 uncapped; a *block call* is one side's execution of Algorithm 1, up to two per iteration), a and b by least squares of the non-column time per iteration on k, c_{col} as column time per column.

Table 14 The timing model fitted per cell (seed 0; the twenty configurations run one after the other in a single process with the per-step instrumentation on, which added 3–25% to the times of September isolated runs of the same seed-0 trajectories at $k = 1$ and 16; the coefficients describe these instrumented runs). $\hat{\eta}$ is the median over $k \in \{4, 16, 32\}$; the $k = 16$ and $k = 32$ estimates agree within 0.02 and the $k = 4$ estimate within 0.04 of them (gisette 0.04/0.08/0.07, w8a 0.20/0.17/0.16). k^* from (3) with $\hat{\eta}$ in place of η; a, b and c_{col} in milliseconds. Times for $k = 1/4/16/32$; "col. share" is the fraction of the k=16 run spent computing columns; the last column is the speed-up of the best k over the guarded single pair.

cell	$\hat{\eta}$	a	b	c_{col}	k^*	time (s), $k = 1/4/16/32$	col. share	best vs k=1
gisette L_2	0.07	1.3	0.080	21.30	14	33.0/29.2/33.9/38.0	93%	1.1× (4)
ijcnn1 KL2	0.26	1.7	0.098	0.23	7	11.6/6.1/6.1/7.6	17%	1.9× (4)
w8a KL2	0.17	1.3	0.077	0.88	9	9.7/5.3/4.4/4.6	64%	2.2× (16)
a9a KL2	0.11	2.5	0.111	0.62	13	30.5/14.2/10.8/11.7	41%	2.8× (16)
covtype KL2	0.29	2.8	0.102	0.35	8	38.9/21.0/18.2/21.7	17%	2.1× (16)

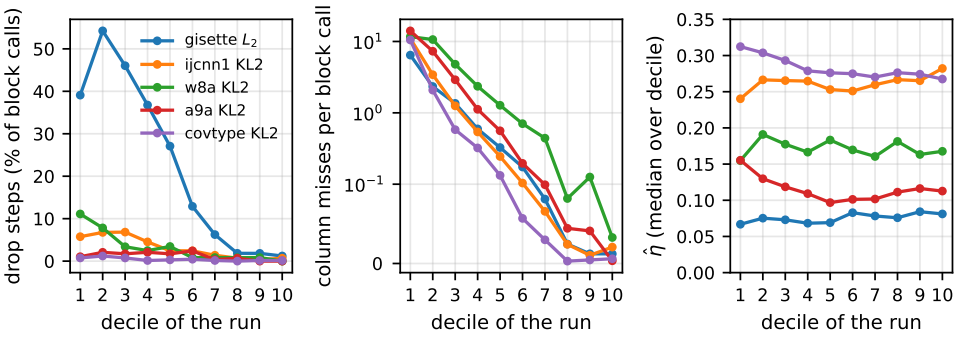


Fig. 4 Along the run at $k = 16$ (deciles of the iteration count, one line per cell): fraction of block calls that are drops, column misses per block call, and the effective correlation $\hat{\eta}$. Drops and misses are paid while the support is built and become rare once it has settled, the shape Theorem 17 describes, most clearly on gisette, ijcnn1 and w8a; $\hat{\eta}$ is nearly constant along the run (a9a falls from 0.16 to 0.10 over the first five deciles, covtype from 0.31 to 0.27).

Four facts follow.

- *$\hat{\eta}$ is stable on each cell.* The effective correlation moves little with k or along the run (Figure 4, right; a9a and covtype drift by 0.04–0.06), although it is a statistic of the iterates and the pairing rule rather than of the data alone; it is 0.07 on gisette, 0.11 on a9a, 0.17 on w8a, 0.26 on ijcnn1 and 0.29 on covtype, and 0.001 on the synthetic instance of Table 16 at d=1,000. The mean relative pair gain $Q/k' = (1/k')\sum_j \Delta_j/\Delta_1$ (median Q over the run's block calls with pair 1 uncapped, divided by the mean number of retained pairs) is 0.92–0.98 at $k = 4$, 0.83–0.98 at $k = 16$ and 0.78–0.88 at $k = 32$ (Table 15 lists the $k = 16$ values): the useful progress beyond the leading pair is never the limit, the loss to combination is. The measured gain ratios (median $\max(\Delta_B, \Delta_1)/\Delta_1$ over the same calls: 3.4/6.9/7.5 on gisette and 2.1/2.8/3.0 on ijcnn1 at $k = 4/16/32$) follow $k/(1+\hat{\eta}(k-1))$ within 35%;

on ijcnn1 they saturate near $1/\hat{\eta} = 3.8$, on gisette they are still rising at $k = 32$ (7.5 against $1/\hat{\eta} = 14$).

- k^* *is* 7–14 *on every cell*, because a/b is 16–28 throughout and $\hat{\eta} \in [0.07, 0.29]$: k=16 is within 16% of the best measured time on all five cells (16% on gisette, under 1% elsewhere; k=4 is within 32% of k=16 on every cell), so the untuned $k = 16$ of Section 8 never lost to another size by more than that.
- *The wall-clock gain is bounded by the single pair's column share.* The block does not reduce the column term of the timing model and on these cells barely raises it (N_{cols} varies by under 20% across k), so no block size can beat the guarded single pair by more than the reciprocal of the fraction of that run spent on columns (T_1/T_{col}: 1.1 on gisette, 2.4 on w8a, 5.4 on a9a, 11–12 on ijcnn1 and covtype). The measured speed-ups are 1.1× on gisette, 2.2× on w8a, 2.8× on a9a and 1.9–2.1× on ijcnn1 and covtype, where the iteration gain has saturated at $1/\hat{\eta}$ long before the column bound matters. (The share of the k=16 run in the table is not a bound: w8a's 2.2× exceeds 1/0.64.) The model's time predictions, $c_{\text{col}}N_{\text{cols}} + T(k)(a + bk)$ with the iteration count $T(k) = T(1)(1 + \hat{\eta}(k-1))/k$ predicted from the single-pair run (`src/model_predictions.py`), are within 13% of the measurements at $k \leq 4$ and within 7% on gisette at every k; at $k = 16$–32 on the kernel cells they are 9–41% too high, because the runs need fewer iterations than the one-state ratio predicts: the trajectory effect is in the block's favour there. On gisette the model predicts too few iterations (457 against 831 at $k = 16$, with drops a third of the calls), but columns dominate the time.
- *Drops are front-loaded* (Figure 4, left). On gisette at $k = 16$ half of the block calls in the first three deciles are drops and 1–2% in the last three; the kernel cells show the same shape at lower levels; most case-(b) calls are drops (78–95%), so the best-of-four branch is rare. Column misses follow the same curve, so on column-bound cells the whole column bill is paid during identification whichever k is used. Total drops per point of one class's final support are 0.13/0.13/0.48/0.69 on gisette and 0.003–0.10 on the kernel cells for $k = 1/4/16/32$: growing with k, and an order of magnitude more frequent on the sparse linear cell, where fresh receivers of small weight become the worst donor at once. On a9a the drop fraction is flat over the first six deciles and on covtype low throughout, rather than front-loaded; the shape of Theorem 17 is clearest on gisette, ijcnn1 and w8a.

The theorem's constants are not what the mechanism runs on. Table 15 records the assumptions of Proposition 15 on the real cells. The maximum pairwise cosine is 0.28–0.42 on every cell, so the η of the proposition does not distinguish the cells at all, whereas the effective correlation $\hat{\eta}$ spans 0.07–0.29 and does; the minimum relative gain is 0.53–0.82; and at $k = 16$ only 43–96% of the block calls have all pairs uncapped, so the proposition applies to a minority of the calls on gisette. The lower bound the proposition guarantees for the guarded ratio from these constants is 1.5–2.2, against measured medians of 2.7–6.9 (the bound is an inequality, so the gap is slack, not a contradiction). The ratio of the medians of Q and χ over the same calls, 7.2/2.9/3.9/4.8/2.8, lies above the measured 6.9/2.8/3.6/4.6/2.7 because it is a ratio of medians over calls that include capped non-leading pairs, where the identity's $S = 2\sum_j \Delta_j$ does not hold; the "model" column, $k'/(1 + \hat{\eta}(k'-1))$, which replaces Q

Table 15 The theorem's assumptions against the measured mechanism at $k = 16$ (seed 0; `src/block_diagnostics.py`, September, on a cloud VM, with iteration and drop counts identical to the runs of Table 14; its $\hat{\eta}$ differs from Table 14 in the second decimal because it is the $k = 16$ median alone rather than the median over $k \in \{4, 16, 32\}$). Over the block calls with every retained pair uncapped: median maximum absolute pairwise cosine of the retained directions (the η of Proposition 15), median minimum relative pair gain (its β_{flat}), and the lower bound the proposition then guarantees for the guarded ratio $\max(\Delta_B, \Delta_1)/\Delta_1$, $(1 + \beta_{\text{flat}}(k' - 1))/(1 + \eta(k' - 1))$ (column "bound"); against the effective correlation $\hat{\eta}$, the mean relative pair gain Q/k', the model ratio $k'/(1 + \hat{\eta}(k' - 1))$ and the measured median of $\max(\Delta_B, \Delta_1)/\Delta_1$, the last three over the block calls with pair 1 uncapped.

cell	all pairs uncapped	max $\lvert\cos\rvert$	β_{flat}	bound	$\hat{\eta}$	Q/k'	model	measured
gisette L_2	43%	0.33	0.53	1.5	0.08	0.98	7.4	6.9
ijcnn1 KL2	73%	0.42	0.80	1.8	0.26	0.89	3.2	2.8
w8a KL2	76%	0.33	0.77	2.1	0.17	0.88	4.4	3.6
a9a KL2	89%	0.28	0.70	2.2	0.12	0.83	5.8	4.6
covtype KL2	96%	0.42	0.82	1.8	0.28	0.91	3.0	2.7

by k', is 7–26% high. Both reproduce the measurement to within the spread because they use the γ-weighted aggregate in which the cross terms largely cancel, not the worst pair. The right reading of Proposition 15 is therefore its first, exact, half; the η-diverse, β_{flat}-flat bound is a worst case that real blocks beat by a factor of 1.5–4.6.

The measurements are from one seed per cell and the fits are retrospective (the coefficients are estimated from the same four runs they describe); the ratios they support (Tables 12 and 13) are reproduced across three seeds in isolated runs, and Section 9.3 is the prospective test. On the identification mechanism the figure is consistent with Theorem 17 but does not verify it: the theorem's thresholds $H_{\text{id}}, H_{\text{w}}$ lie below the stopping tolerance of these runs, and late drops, though rare, do occur. Section 9.4 tests the theorem where its constants can be computed.

9.3 An adaptive block size

Every quantity in (3) is observable while the solver runs: χ is computed by the guard at every block step, so $\hat{\eta} = (\chi - 1)/(k' - 1)$ costs nothing, and a/b is a property of the implementation (16–28 in the September instrumented runs of Table 14). The released code therefore offers `block_size='auto'`: every 50 iterations it sets k to $\sqrt{(a/b)(1 - \hat{\eta})/\hat{\eta}}$, clipped to $[1, k_{\max}]$, with $\hat{\eta}$ the median over the window's block steps with $k' > 1$ (when fewer than five such steps exist, as at $k = 1$, the rule doubles k to obtain a signal; a non-positive $\hat{\eta}$ is floored at 10^{-3} before the square root, which sends k to $k_{\max}$), and halves k instead when more than 10% of the window's block calls had their leading pair capped (drops are outrunning the support). Guard and fallback are untouched, so Theorem 6 applies with $k_{\max}$ in the drop bound. The rule's form, its window, $k_{\max} = 32$ and the ratio $a/b = 20$ were set from the analysis of the two pilot cells (gisette, ijcnn1); on a run the rule reads only its own window's χ values, never the per-cell $\hat{\eta}$ of Table 14, which is a retrospective median over three complete runs and enters nothing but the fitted k^* column of that table. We do not call the other three cells a prospective test: the k-sweeps on w8a, a9a and covtype were on disk before the

Table 16 Adaptive block size on synthetic L_2 instances of varying directional diversity (one process, seed 0, a/b set to 20, $k_{\max} = 32$). Times for guarded k=1 and fixed $k = 4/16/32$; "auto" reports the size the rule settled on and its time.

d	$\hat{\eta}$	k^*	k=1 (s)	k=4	k=16	k=32	auto: k, time (s)
20	0.31	7	1.3	1.0	1.1	1.3	6, 1.1
50	0.10	13	2.1	1.2	1.0	1.3	12, 1.1
100	0.03	24	3.5	1.8	1.1	1.2	24, 1.1
300	0.00	> 32	1.7	0.7	0.4	0.4	32, 0.4
1,000	0.00	> 32	1.5	0.9	0.7	0.7	32, 0.7

rule was committed, and the repository cannot establish that the constants were fixed without sight of them. They are cells not used to set the constants, and the check is whether the rule's wall-clock on them matches the fixed sizes run under the same protocol (Table 12). It does: the rule settles at $k = 10$, 13 and 7 (the retrospectively fitted k^* are 9, 13, 8) and takes 2.5 ± 0.1, 7.0 ± 0.3 and 12.4 ± 0.8 s against 2.5 ± 0.2, 6.9 ± 0.6 and 12.2 ± 0.2 s for k=16, a tie within CI on every cell, as it is on the two pilot cells (k=9 then 15 on gisette; k=7–8 on ijcnn1, 3.7 ± 0.1 against 3.8 ± 0.3 s). The rule's value is that it removes the parameter, not that it beats a well-chosen fixed size. Table 16 tests it where η actually varies: synthetic L_2 instances (n=2,000 per class, C=1) whose transfer directions decorrelate as the dimension grows, so that $\hat{\eta}$ falls from 0.31 at d=20 to below 0.002 at $d \geq 300$. The rule tracks k^* across two orders of magnitude of $\hat{\eta}$ and is within 0.14 s (14%) of the best fixed size at d=20 and within 0.1 s elsewhere, the resolution of these short single-seed runs.

Robustness and safeguards. The rule reads $\hat{\eta}$ over a moving window, so it follows the phases of a run; on the real cells $\hat{\eta}$ is nearly constant along the run (Figure 4, right). Its safeguards are the clip to $[1, k_{\max}]$, the floor on $\hat{\eta}$, and the halving when leading pairs are capped, which is the signal that the block adds indices faster than drops remove them. It has no safeguard against column misses: the rule ignores c_{col}, because on the real cells N_{cols} varies by under 20% across k, so a large k costs little there. Where a larger block does pull in many new receivers, as on the synthetic d=100 L_2 instance of Table 9 (columns grow from 1,345 to 3,076 and single MDM is fastest), the rule would choose too large a k; it was not run on that instance. A miss-aware rule would add the measured $c_{\text{col}} n_B^{\text{new}}$ to the cost side of (2).

9.4 Numerical check of Theorem 17

The theorem's constants are computable on small instances. For 40 random hard-margin instances ($d = 4$, ten points per class, Gaussian clouds 2.5 apart) the SLSQP reference gives z^*, the faces, τ, $\alpha_{\min}$, $d_{\min}$ and σ (as the singular value of Assumption 16); 12 instances are skipped, 8 whose hulls intersect ($\delta^* = 0$) and 4 that fail strict complementarity within 10^{-7} (no instance tested for affine independence fails it; the four that fail strict complementarity are not tested), and the remaining 28 are run with $k \in \{1, 4, 16\}$ to certificate tolerance $\varepsilon = 10^{-10}$ with both supports snapshotted at every iteration (`src/ident_check.py`, 84 runs). Three things are checked. The weight inequality (F2), $\omega_t \leq h_t/\tau$, holds at every iteration of every run. From the

first iteration at which the proof's conditions hold (the conditions on $\sqrt{2h_t}$, ω_t and $2h + R\sqrt{2h}$ that define H_{id} and H_{w}), claims (i) and (ii) hold to the end of every run: every removal of a non-face point is a drop, no non-face point enters, and after t_1 no index leaves. The third finding is how much earlier the mechanism engages than the theorem certifies it. The last non-face support point leaves, and the last drop occurs, at $h \approx 10^{-3}$ (median; $h_0 \approx 6$); the proof's conditions first hold at $h \approx 1.6 \times 10^{-7}$, and the thresholds $H_{\mathrm{id}}, H_{\mathrm{w}}$ lie at 5.5×10^{-7} (both medians over runs, so they need not be ordered): the theorem is verified in its ingredients and contradicted nowhere, and its thresholds are pessimistic by about 3.5 orders of magnitude in h on these instances, so that on every run the non-face points are already gone when t_{id} arrives and the drop part of claim (i) holds vacuously; the check exercises claim (ii) and the weight inequality non-vacuously. Passing the distance bound through the screening radius instead, as the workshop version did, puts the same thresholds at 2.5×10^{-18}, eleven orders lower; the direct bound (4) is what makes the thresholds computable in practice. The large-data profiles of Figure 4 are consistent with the mechanism; they do not verify the thresholds, which no run of Section 8 reaches.

10 Conclusion

Guarded block transfers give the Triangle Algorithm a linear rate with constants explicit in the pyramidal width of the difference atom set and no combinatorial swap-step bounds, and a safe screening rule with a finite activation bound. The experiments, rerun with full baseline caches and both LIBLINEAR solvers, locate the boundary and narrow the method's niche. Where the augmented hulls are well separated (gisette in every formulation; the kernel L_2 margins, whose supports are dense) the solver ties LIBSVM on the linear hard margin, is 1.2× faster on the RBF hard margin, beats our column-oracle BPCG by four to seven times on the four non-gisette kernel L_2 cells and supplies a certificate; but a full-cache SMO ties or beats it on every kernel L_2 margin, LIBSVM with the kernel in memory was faster still in September, and LIBLINEAR's dual solver solves the separable L_2 margin up to 4.5 times faster. Where the hulls nearly touch or the support is dense in the linear case, LIBLINEAR's Newton solver or LIBSVM is 30 to over 2,000 times faster. What the solver offers that these baselines do not is the certified interval and the TA I intersection test, not speed. Section 6 answers the question the guard raises: aggregation acts on the per-iteration term of the work by the exact factor $Q(2 - \chi)$ or Q/χ per step; drops are an identification cost that becomes rare once the optimal faces are found and stops in the regime of Theorem 17; and a plug-in rule, $k = \sqrt{(a/b)(1 - \hat{\eta})/\hat{\eta}}$ with the measured $\hat{\eta}$ in place of the theorem's η, sets the block size from the χ the guard already computes; on column-bound problems no block size helps.

Limitations and open problems.

Screening pays only when the certificate closes early relative to the scan's share of the iteration, which on the real cells here it does not. The implementation is NumPy-level and dense. The rate constant is explicit in $\delta = \mathrm{PW}(A)$, which is neither computed nor bounded in terms of the class hulls, and cannot be in general (Section 2); computing

the facial distance of Peña and Rodríguez (2019) on the small instances of Section 9.4 and comparing it with the observed contraction would show how loose the 1/16 is. The ν-SVM variant is not analysed. No corrective baseline was run: fully-corrective FW, Wolfe's method and the quadratic corrections of Halbey et al. (2025), under the same certificate and tolerance, would show whether re-optimising over the support beats aggregating transfers on these problems, and Remark 10 gives the guarded combination of the two. Approximate oracles would act on the wrong term or the wrong problem. Lazy or prioritised receiver search (Braun, Pokutta, & Zink, 2019), which the implementation already uses between full scans, saves scans; but the lower bound LB needs the exact minimum over all points, so every certificate still needs a full scan. The scan is also already a small share of the time (0.1–0.7 s of the k=16 runs in Table 12; the per-iteration bill is the score update and the certificate), and the column misses are paid during identification whatever the step (Figure 4). Low-rank or random-feature kernel approximations change the objective, so the certificate would bound the distance of the approximate problem only. Two comparisons remain confounded by implementation or machine: BPCG ran on the product-atom parametrisation (a side-transfer BPCG would isolate the step rule), and the LIBSVM-precomputed baseline and covtype's ETA and SMO entries come from the September machine (a full-cache SMO on covtype needs about 80 GB). The October rerun resolved the other two September caveats, both against the method: with a full row cache our SMO computes as many rows as K-ETA computes columns, which removed K-ETA's apparent kernel-column advantage, and LIBLINEAR's dual solver, swept across tolerances, is the fastest solver on the gisette L_2 margin. Assumption 16 fails when a class contains duplicate points (w8a does) or when parallel optimal faces make the witness pair non-unique; for the L_2 augmentation and strictly positive definite kernels the affine independence part holds automatically.

Funding. This work was self-funded; no external funding was received.

Competing interests. The author is an independent researcher and has no competing interests to declare.

Ethics approval. Not applicable.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. The datasets analysed are the public LIBSVM benchmark sets a9a, w8a, ijcnn1, gisette and covtype and the 5,000-image MNIST subset of the mlxtend package (https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/); the synthetic instances are generated by the released code from fixed seeds.

Code availability. https://github.com/iammayankg/triangle-algorithm-sv (archived release doi:10.5281/zenodo.23304507); the scripts behind each table and figure are named in Section 8.3.

Authors' contributions. Single author.

Use of AI tools. Large language models (Claude, Anthropic) were used to draft code, experiment scripts and text, as stated in Section 8.3; the author ran and verified every result and is responsible for the content.

Prior publication. A version of this work (six pages of main text with appendices, twenty-one pages in all) was accepted as a poster at the OPT 2026 workshop (NeurIPS 2026), which publishes no archival proceedings; its camera-ready version shares the October rerun of the timing experiments. New in this article: the prose related work with fourteen added references, the work bound in standard form (Corollary 13), the notation table, Sections 6 and 9 (exact gain ratio, identification theorem, conditional iteration bound and timing model, per-step measurements on five real cells, and the adaptive block size with its synthetic sweep), the adaptive-solver rows of Table 12, the screening and n-scaling experiments on real and synthetic data, and the identification check. The algorithm, its analysis, the reductions and the regime batteries appeared in the workshop version.

References

Beck, A., & Shtern, S. (2017). Linearly convergent away-step conditional gradient for non-strongly convex functions. *Mathematical Programming*, *164*(1–2), 1–27, https://doi.org/10.1007/s10107-016-1069-4

Bennett, K.P., & Bredensteiner, E.J. (2000). Duality and geometry in SVM classifiers. *Proceedings of the 17th international conference on machine learning (icml)* (pp. 57–64). Morgan Kaufmann.

Besançon, M., Pokutta, S., Wirth, E. (2025). The pivoting framework: Frank–Wolfe algorithms with active set size control. *Proceedings of the 28th international conference on artificial intelligence and statistics (aistats)* (Vol. 258, pp. 271–279).

Bomze, I.M., Rinaldi, F., Zeffiro, D. (2020). Active set complexity of the away-step Frank–Wolfe algorithm. *SIAM Journal on Optimization*, *30*(3), 2470–2500, https://doi.org/10.1137/19M1309419

Braun, G., Carderera, A., Combettes, C.W., Hassani, H., Karbasi, A., Mokhtari, A., Pokutta, S. (2025). *Conditional gradient methods*. Philadelphia: SIAM. (Earlier version: arXiv:2211.14103 (2022))

Braun, G., Pokutta, S., Zink, D. (2019). Lazifying conditional gradient algorithms. *Journal of Machine Learning Research*, *20*(71), 1–42,

Chang, C.-C., & Lin, C.-J. (2011). LIBSVM: a library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, *2*(3), 27:1–27:27,

https://doi.org/10.1145/1961189.1961199

Clarkson, K.L. (2010). Coresets, sparse greedy approximation, and the Frank–Wolfe algorithm. *ACM Transactions on Algorithms*, *6*(4), 63:1–63:30, https://doi.org/10.1145/1824777.1824783

Combettes, C.W., & Pokutta, S. (2020). Boosting Frank–Wolfe by chasing gradients. *Proceedings of the 37th international conference on machine learning (icml)* (Vol. 119, pp. 2111–2121).

Crisp, D.J., & Burges, C.J.C. (2000). A geometric interpretation of ν-SVM classifiers. *Advances in neural information processing systems* (Vol. 12, pp. 244–250).

El Ghaoui, L., Viallon, V., Rabbani, T. (2012). Safe feature elimination in sparse supervised learning. *Pacific Journal of Optimization*, *8*(4), 667–698,

Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R., Lin, C.-J. (2008). LIBLINEAR: a library for large linear classification. *Journal of Machine Learning Research*, *9*, 1871–1874,

Fercoq, O., Gramfort, A., Salmon, J. (2015). Mind the duality gap: safer rules for the Lasso. *Proceedings of the 32nd international conference on machine learning (icml)* (Vol. 37, pp. 333–342).

Franc, V., & Hlaváč, V. (2003). An iterative algorithm learning the maximal margin classifier. *Pattern Recognition*, *36*(9), 1985–1996, https://doi.org/10.1016/S0031-3203(03)00060-8

Frank, M., & Wolfe, P. (1956). An algorithm for quadratic programming. *Naval Research Logistics Quarterly*, *3*(1–2), 95–110, https://doi.org/10.1002/nav.3800030109

Garber, D. (2020). Revisiting Frank–Wolfe for polytopes: strict complementarity and sparsity. *Advances in neural information processing systems* (Vol. 33, pp. 18883–18893).

Garber, D., & Hazan, E. (2016). A linearly convergent variant of the conditional gradient algorithm under strong convexity, with applications to online and stochastic optimization. *SIAM Journal on Optimization*, *26*(3), 1493–1528, https://doi.org/10.1137/140985366

Garber, D., & Wolf, N. (2021). Frank–Wolfe with a nearest extreme point oracle. *Proceedings of the 34th conference on learning theory (colt)* (Vol. 134, pp. 2103–2132).

Gärtner, B., & Jaggi, M. (2009). Coresets for polytope distance. *Proceedings of the 25th annual symposium on computational geometry (socg)* (pp. 33–42).

Gilbert, E.G. (1966). An iterative procedure for computing the minimum of a quadratic form on a convex set. *SIAM Journal on Control*, *4*(1), 61–80, https://doi.org/10.1137/0304007

Guélat, J., & Marcotte, P. (1986). Some comments on Wolfe's 'away step'. *Mathematical Programming*, *35*(1), 110–119, https://doi.org/10.1007/BF01589445

Gupta, M., & Kalantari, B. (2026). An enhanced triangle algorithm for large-scale support vector machine optimization: a comparative study with classical and modern solvers. *Journal of Intelligent Decision Making and Information Science*, *3*(9s), 1–15,

Halbey, J., Rakotomandimby, S., Besançon, M., Designolle, S., Pokutta, S. (2025). Efficient quadratic corrections for Frank–Wolfe algorithms. *Advances in neural information processing systems 38 (neurips).*

Hsieh, C.-J., Chang, K.-W., Lin, C.-J., Keerthi, S.S., Sundararajan, S. (2008). A dual coordinate descent method for large-scale linear SVM. *Proceedings of the 25th international conference on machine learning (icml)* (pp. 408–415).

Jaggi, M. (2013). Revisiting Frank–Wolfe: projection-free sparse convex optimization. *Proceedings of the 30th international conference on machine learning (icml)* (Vol. 28(1), pp. 427–435).

Kalantari, B. (2015). A characterization theorem and an algorithm for a convex hull problem. *Annals of Operations Research*, *226*(1), 301–349, https://doi.org/10.1007/s10479-014-1707-2

Kalantari, B. (2019). An algorithmic separating hyperplane theorem and its applications. *Discrete Applied Mathematics*, *256*, 59–82, https://doi.org/10.1016/j.dam.2018.05.009

Keerthi, S.S., Shevade, S.K., Bhattacharyya, C., Murthy, K.R.K. (2000). A fast iterative nearest point algorithm for support vector machine classifier design. *IEEE Transactions on Neural Networks*, *11*(1), 124–136, https://doi.org/10

.1109/72.822516

Lacoste-Julien, S., & Jaggi, M. (2015). On the global linear convergence of Frank–Wolfe optimization variants. *Advances in neural information processing systems* (Vol. 28, pp. 496–504).

Lacoste-Julien, S., Jaggi, M., Schmidt, M., Pletscher, P. (2013). Block-coordinate Frank–Wolfe optimization for structural SVMs. *Proceedings of the 30th international conference on machine learning (icml)* (Vol. 28(1), pp. 53–61).

López, J., & Dorronsoro, J.R. (2015). Linear convergence rate for the MDM algorithm for the nearest point problem. *Pattern Recognition*, *48*(4), 1510–1522, https://doi.org/10.1016/j.patcog.2014.10.015

Mavroforakis, M.E., & Theodoridis, S. (2006). A geometric approach to support vector machine (SVM) classification. *IEEE Transactions on Neural Networks*, *17*(3), 671–682, https://doi.org/10.1109/TNN.2006.873281

Mitchell, B.F., Demyanov, V.F., Malozemov, V.N. (1974). Finding the point of a polyhedron closest to the origin. *SIAM Journal on Control*, *12*(1), 19–26, https://doi.org/10.1137/0312003

Ñanculef, R., Frandi, E., Sartori, C., Allende, H. (2014). A novel Frank–Wolfe algorithm. Analysis and applications to large-scale SVM training. *Information Sciences*, *285*, 66–99, https://doi.org/10.1016/j.ins.2014.03.059

Ndiaye, E., Fercoq, O., Gramfort, A., Salmon, J. (2017). Gap safe screening rules for sparsity enforcing penalties. *Journal of Machine Learning Research*, *18*(128), 1–33,

Ogawa, K., Suzuki, Y., Takeuchi, I. (2013). Safe screening of non-support vectors in pathwise SVM computation. *Proceedings of the 30th international conference on machine learning (icml)* (Vol. 28(3), pp. 1382–1390).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830,

Peña, J., & Rodríguez, D. (2019). Polytope conditioning and linear convergence of the Frank–Wolfe algorithm. *Mathematics of Operations Research*, *44*(1), 1–18,

https://doi.org/10.1287/moor.2017.0910

Peña, J., Rodríguez, D., Soheili, N. (2016). On the von Neumann and Frank–Wolfe algorithms with away steps. *SIAM Journal on Optimization*, *26*(1), 499–512, https://doi.org/10.1137/15M1009937

Platt, J.C. (1999). Fast training of support vector machines using sequential minimal optimization. B. Schölkopf, C.J.C. Burges, & A.J. Smola (Eds.), *Advances in kernel methods: Support vector learning* (pp. 185–208). MIT Press.

Rinaldi, F., & Zeffiro, D. (2023). Avoiding bad steps in Frank–Wolfe variants. *Computational Optimization and Applications*, *84*(1), 225–264, https://doi.org/10.1007/s10589-022-00434-3

Shibagaki, A., Karasuyama, M., Hatano, K., Takeuchi, I. (2016). Simultaneous safe screening of features and samples in doubly sparse modeling. *Proceedings of the 33rd international conference on machine learning (icml)* (Vol. 48, pp. 1577–1586).

Tsang, I.W., Kwok, J.T., Cheung, P.-M. (2005). Core vector machines: fast SVM training on very large data sets. *Journal of Machine Learning Research*, *6*, 363–392,

Tsuji, K.K., Tanaka, K., Pokutta, S. (2022). Pairwise conditional gradients without swap steps and sparser kernel herding. *Proceedings of the 39th international conference on machine learning (icml)* (Vol. 162, pp. 21864–21883).

Wang, J., Wonka, P., Ye, J. (2014). Scaling SVM and least absolute deviations via exact data reduction. *Proceedings of the 31st international conference on machine learning (icml)* (Vol. 32(2), pp. 523–531).

Wolfe, P. (1976). Finding the nearest point in a polytope. *Mathematical Programming*, *11*(1), 128–149, https://doi.org/10.1007/BF01580381